

1

Unidad I y II : Introducción a la Arquitectura de Computadoras

Sistema de Numeración

¿Qué es un Sistema de Numeración?

Es un:

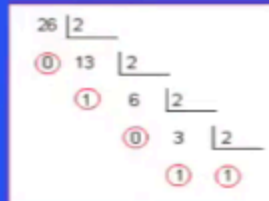
- Un sistema de numeración en *base b* utiliza para representar los números un alfabeto compuesto por **b** símbolos o cifras
- Ejemplos:
 - $b = 10$ (*decimal*) $\{0,1,2,3,4,5,6,7,8,9\}$
 - $b = 16$ (*hexadecimal*) $\{0,1,2,3,4,5,6,7,8,9,A,B,C,D,E,F\}$
 - $b = 2$ (*binario*) $\{0,1\}$
- El número se expresa mediante una secuencia de cifras:
$$N \equiv \dots n_4 n_3 n_2 n_1 n_0 n_{-1} n_{-2} n_{-3} \dots$$
- El valor de cada cifra depende de la cifra en sí y de la posición que ocupa en la secuencia

Conversión decimal - base b

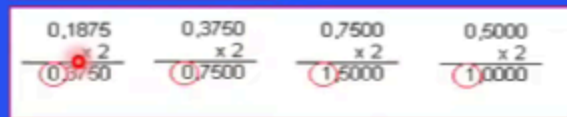
- Método de divisiones sucesivas entre la base b
- Para números fraccionarios se realizan multiplicaciones sucesivas por la base b.
- Consideración de restos mayores que 9 y Error de truncamiento

Ejemplos:

$$26_{10} = 11010_2$$



$$0,1875_{10} = 0,0011_2$$



$$26,1875_{10} = 11010,0011_2$$

- Rango de representación: Conjunto de valores representable. Con n cifras en la base b podemos formar b^n combinaciones distintas. $[0..b^n-1]$
- Sistema de numeración en base dos o binario

$b = 2$ (binario) $\{0,1\}$

Números binarios del 0 al 7

Decimal	Binario
0	000
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Ejemplos:

$$1101002 = (1 \cdot 2^5) + (1 \cdot 2^4) + (1 \cdot 2^2) = 25 + 24 + 22 = 32 + 16 + 4 = 52_{10}$$

$$0,101002 = 2^{-1} + 2^{-3} = (1/2) + (1/8) = 0,625_{10}$$

$$10100,0012 = 2^4 + 2^2 + 2^{-3} = 16 + 4 + (1/8) = 20,125_{10}$$

¿A que es equivalente la resta de dos números binarios?

Es equivalente a la suma entre el primer elemento y el complemento del segundo. Esto se conoce como complemento a la base menos uno.

HEX. N°		0		1		2		ZONA		6	7
HEX. N°	BITS 654 3210	000	001	010	011	100	101	110	111		
0	0000	NUL	DLE	SP	0	@	P	`	p		
1	0001	SOH	DCI	!	1	A	Q	a	q		
2	0010	STX	DC2	"	2	B	R	b	r		
				#	3	C	S	c	s		
				\$	4	D	T	d	t		
				%	5	E	U	e	u		
6	0110	ACK	SYN	&	6	F	V	f	v		
7	0111	BEL	ETB	'	7	G	W	g	w		
8	1000	BS	CAN	(	8	H	X	h	x		
9	1001	HT	EM	)	9	I	Y	i	y		
A	1010	LF	SUB	*	:	J	Z	j	z		
B	1011	VT	ESC	+	:	K	[	k	{		
C	1100	FF	FS	,	<	L	\	l			
D	1101	CR	GS	-	=	M	]	m	}		
E	1110	SO	RS	.	>	N	^	n	~		
F	1111	SI	US	/	?	O	_	o	DEL		

Las computadoras trabajan exclusivamente con datos binarios (ceros y unos). Cuando un usuario ingresa un texto (por ejemplo, "A"), la computadora lo traduce a un valor numérico según esta tabla (en este caso, "A" corresponde al código 0100 0001 o 0x41 en hexadecimal).

La tabla ASCII asegura que los sistemas informáticos puedan interpretar los mismos caracteres de forma consistente. Por ejemplo, una "A" ingresada en un teclado se traducirá al mismo valor binario, sin importar el dispositivo.

ASCII le destinaba tres bits a la zona, cuatro bits al dígito, y uno para la paridad.

¿Cómo sería el flujo de procesamiento de una letra?

Por ejemplo, si presionas la tecla "A", el teclado detecta esa acción y envía una señal eléctrica correspondiente a esa tecla.

Internamente, el teclado tiene un pequeño chip que sabe que la letra "A" corresponde al número 65 en el código ASCII (que en binario es 0100 0001). Ese número se envía a la computadora.

La señal pasa por el controlador de teclado, que traduce la información del hardware en datos que la computadora pueda usar. Este controlador toma el código ASCII de "A" y lo almacena temporalmente en un registro o memoria del sistema.

El sistema operativo (Windows, Linux, etc.) recibe el valor 0100 0001 (65 en decimal) y lo interpreta como la letra "A". Luego, decide qué hacer con ese dato según el contexto:

Dependiendo de la aplicación:

- Puede mostrar el carácter en pantalla (como texto en un documento).
- Puede guardarlo en la memoria o realizar una operación específica (por ejemplo, buscar la letra "A" en un archivo).

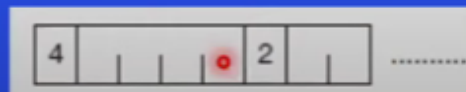
¿Hoy en día se siguen utilizando las tablas ASCII Y EBCDI?

No, hoy en día, Unicode es el estándar más utilizado en el mundo. Prácticamente todas las aplicaciones modernas, sistemas operativos, navegadores web y dispositivos lo emplean. Es la base para que puedas escribir y leer textos en cualquier idioma o usar símbolos y emojis sin problemas.

¿Cómo es la delimitación de Strings en ASCII?

Las cadenas de caracteres alfanuméricos (strings), procesadas y almacenadas en la computadora, pueden ser:

- **De longitud fija:** cada dato ocupa un número determinado de bytes fijo. Por lo tanto, el comienzo y el final de cada dato puede conocerse con rapidez.
- **De longitud variable:** Cada dato tiene en su inicio un campo en el que se indica la longitud en bytes, como lo muestra a continuación.



¿Cuál es la principal diferencia entre el ASCII y el BCD?

El BCD está diseñado específicamente para representar números decimales de manera más directa y eficiente, mientras que el ASCII es un estándar general para representar caracteres alfanuméricos. Los puntos clave serían:

- **Representación directa de números:** En BCD, cada dígito decimal (0 a 9) se representa con exactamente 4 bits (un nibble). Por ejemplo:

El número 9 en BCD es 1001.

En ASCII, los números también pueden representarse, pero requieren 7 u 8 bits por cada carácter, y no son directamente procesables como números. Por ejemplo:

El número 9 en ASCII es 00111001 (representa el carácter '9', no el valor numérico 9).

- **Reducción del espacio para datos numéricos:** Como el BCD usa 4 bits por dígito, ocupa menos espacio que ASCII, que usa 8 bits para cada dígito numérico (por ejemplo, el '9' en ASCII ocupa 8 bits mientras que en BCD ocupa solo 4).

¿Qué aplicaciones tiene el BCD en el campo de la Computación?

Poseen dos aplicaciones:

DESEMPAQUETADO: Los números se codifican utilizando un Byte por dígito (zona y dígito) y es utilizado para transmisión desde hacia periféricos del computador

Supongamos que quieres representar el número 25. En BCD desempaqueado, se vería así:

Dígito 2: Se guarda como 00100000 (un byte donde la zona es 0010 y el dígito es 0000).

Dígito 5: Se guarda como 01010000 (la zona es 0101 y el dígito es 0000).

EMPAQUETADO: Para el tratamiento de cálculos numéricos la ALU no necesita la ZONA, por lo que la elimina, reduciendo la codificación a solo 4 Bits por dígito rellenando con unos los de orden superior. Se los utiliza para cálculos en el entorno de la ALU. El signo se lo incluye por lo general en la parte sobrante del octeto (cabecera o cola del número). Se agrega un cero adelante para mantener la paridad de la cadena en caso de ser necesaria.

Usando el mismo número 25, en BCD empaquetado se vería como 00100101 (los primeros 4 bits representan el 2, y los últimos 4 bits representan el 5).

¿Qué es el BCD Exceso en Tres?

El BCD en exceso tres (o BCD exceso-3) es un sistema de codificación binaria que se utiliza para representar números decimales, pero con una particularidad: a cada dígito decimal se le suma 3 antes de convertirlo a binario. Esta codificación es conocida como "código autocomplementario", porque el complemento restringido (inversión de los bits) de un número en este código representa su complemento decimal respecto a 9.

Valor decimal	BCD exceso tres
0	0011
1	0100
2	0101
3	0110
4	0111
5	1000
6	1001
7	1010
8	1011
9	1100

"Código autocomplementario"

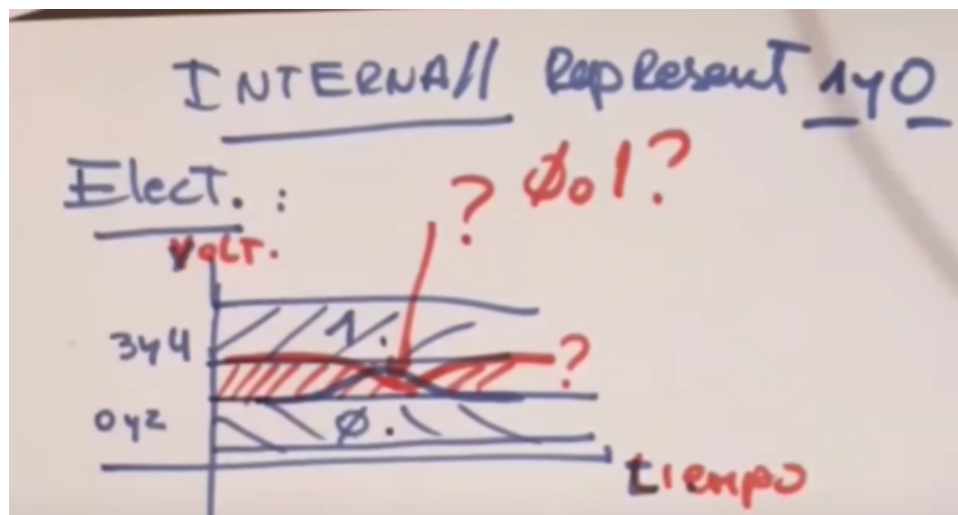
¿Qué es el BCD AIKEN o 2421?

El código BCD 2421 (también conocido como código AIKEN) es un sistema de codificación decimal binaria en el cual a cada dígito decimal se le asigna un código de 4 bits con pesos específicos: 2, 4, 2, 1. Este sistema es diferente al BCD estándar (8421), ya que la suma de los pesos de los bits encendidos (valores de 1) debe dar como resultado el valor decimal representado.

Valor decimal	BCD 2421	
0	0000	"Código autocomplementario"
1	0001	
2	0010	
3	0011	
4	0100	
5	1011	
6	1100	
7	1101	
8	1110	
9	1111	

¿Cómo interpretan las computadoras la entrada de un 0 o un 1?

Las computadoras trabajan con 1 y 0, que son como su idioma nativo. Estos valores son llamados bits, y la máquina los representa usando señales eléctricas. En vez de ver 1 y 0 como números, pensémoslo como que son niveles de voltaje.

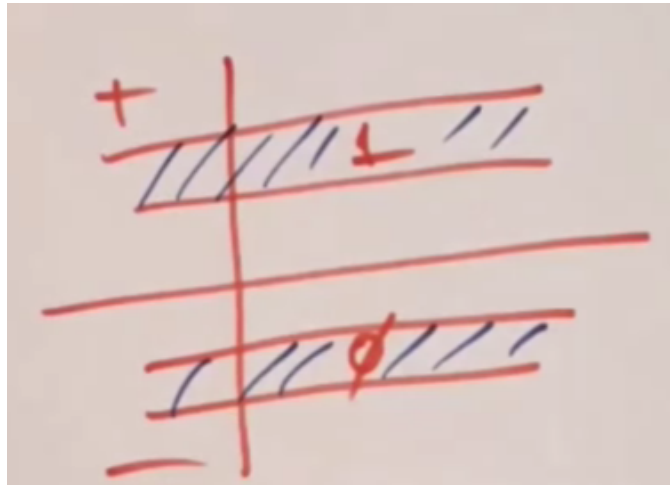


El voltaje puede variar un poco por ruido o errores, pero se establecen rangos para interpretar los valores. Por ejemplo:

- Voltaje entre 0 y 2V → Es un 0.
- Voltaje entre 4 y 4V → Es un 1.

Las áreas en rojo representan zonas donde la señal no está claramente en 0 o 1. Esto puede pasar por interferencia eléctrica o problemas en el hardware, los circuitos están diseñados para filtrar este ruido y asegurarse de interpretar la señal correctamente.

Lo que se hace, es trabajar en dos rangos de representación distinta, es decir, en el eje positivo, y en el eje negativo.



Sistema de Comunicación

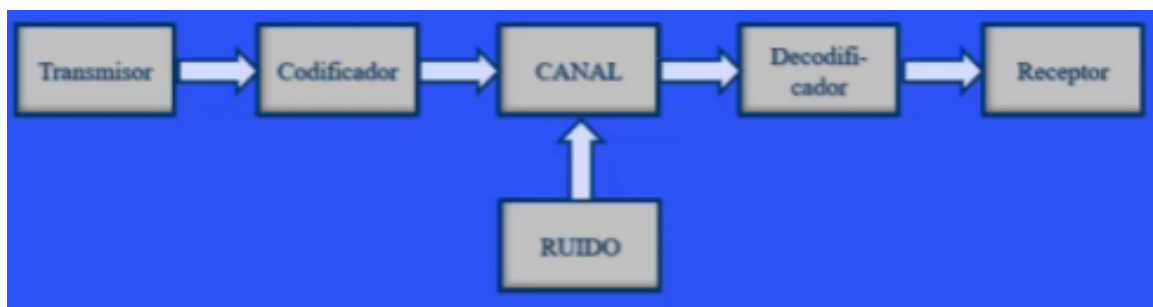
¿Cómo esta compuesto un Sistemas de Comunicación Ideal?

Un Sistema de Comunicación ideal esta compuesto por tres partes:

- Transmisor, remitente o fuente
- Canal o medio de almacenamiento
- Receptor

¿Cómo funciona la comunicación de señales en un sistema digital o analógico?

Podemos esquematizar su funcionamiento de la siguiente manera:



- Transmisor: Es el punto de partida. Aquí se genera el mensaje original. Puede ser algo analógico (como tu voz en un micrófono) o algo digital (como un archivo que querés enviar por Internet).

Si el mensaje es analógico, pasa por un proceso llamado muestreo y digitalización para convertirlo en un formato binario que la computadora pueda manejar. Por ejemplo, una palabra como "Hola" se convierte en una serie de bits: 01001000 01101111 01101100 01100001.

- Codificación: Aquí entra la protección del mensaje, la información original se convierte en un formato más adecuado para el canal de transmisión. Esto incluye:
 - **Compresión**: Para hacer el mensaje más pequeño (por ejemplo, un archivo ZIP o MP3).
 - **Codificación redundante**: Agregar información extra (bits redundantes) para proteger el mensaje contra errores. Esto asegura que si el ruido altera el mensaje, el receptor pueda detectar y/o corregir esos errores. Algunos de los códigos redundantes utilizados en esta etapa son:
 1. **Bit de paridad**: Agrega 1 bit para verificar si el número de bits "1" es par o impar. Detecta errores simples, pero no los corrige.
 2. **Códigos de Hamming**: Permiten detectar y corregir errores de hasta 1 bit.
 3. **Códigos CRC (Cyclic Redundancy Check)**: Muy usados en redes y discos duros. Detectan errores en bloques de datos grandes usando matemáticas más complejas.
 4. **Reed-Solomon**: Usado en CDs, DVDs y comunicaciones inalámbricas. Corrige errores causados por ruido en ráfagas (cuando varios bits consecutivos son afectados).

El resultado de esta etapa es el mensaje original se transformado en algo más robusto, con información adicional para resistir el ruido.

- Canal: Es un medio de comunicación entre un emisor y un receptor, que permite transmitir información de extremo a extremo, minimizando la tasa de errores producto del medio, a través de los sistemas de codificación redundante. Puede ser Alámbrico, es decir, Cable coaxial, fibra óptica o Inalámbrico, o sea, Ondas de radio, WiFi, satélites. fu

El canal presenta los siguientes problemas:

1. **Ruido:**

- Interferencias electromagnéticas.
- Variaciones en el voltaje.
- Señales de otros dispositivos cercanos.

2. **Atenuación:**

- La señal pierde fuerza con la distancia.

3. **Distorsión:**

- Cambios en la forma de la señal debido a las características del canal.

Cuando se presentan dichos problemas, los códigos redundantes aseguran que el mensaje pueda ser reconstruido más adelante.

- **Ruido:** Es cualquier interferencia no deseada que distorsiona la señal original. Por ejemplo, en una llamada telefónica, el ruido puede ser un zumbido que hace que pierdas claridad.
 - Tipos de ruido:
 - **Térmico:** Generado por la resistencia eléctrica de los materiales.
 - **Interferencia electromagnética:** De otros dispositivos o radiación externa.
 - **Ruido de cuantización:** Al convertir señales analógicas a digitales, se pierden detalles.

Su Impacto en la señal puede verse como el cambio de un bit de 1 a 0, o viceversa. Y como pérdidas en bloques enteros de datos.

- **Decodificación:** El receptor usa los códigos redundantes para:
 1. **Detectar errores:** Identificar si el mensaje fue alterado.
 2. **Corregir errores:** En caso de que el código permita corrección, arregla los bits dañados.
- **Receptor:** Es el destino final del mensaje, toma los datos decodificados, elimina los bits redundantes, y reconstruye el mensaje original para entregarlo al usuario.

Códigos Redundantes

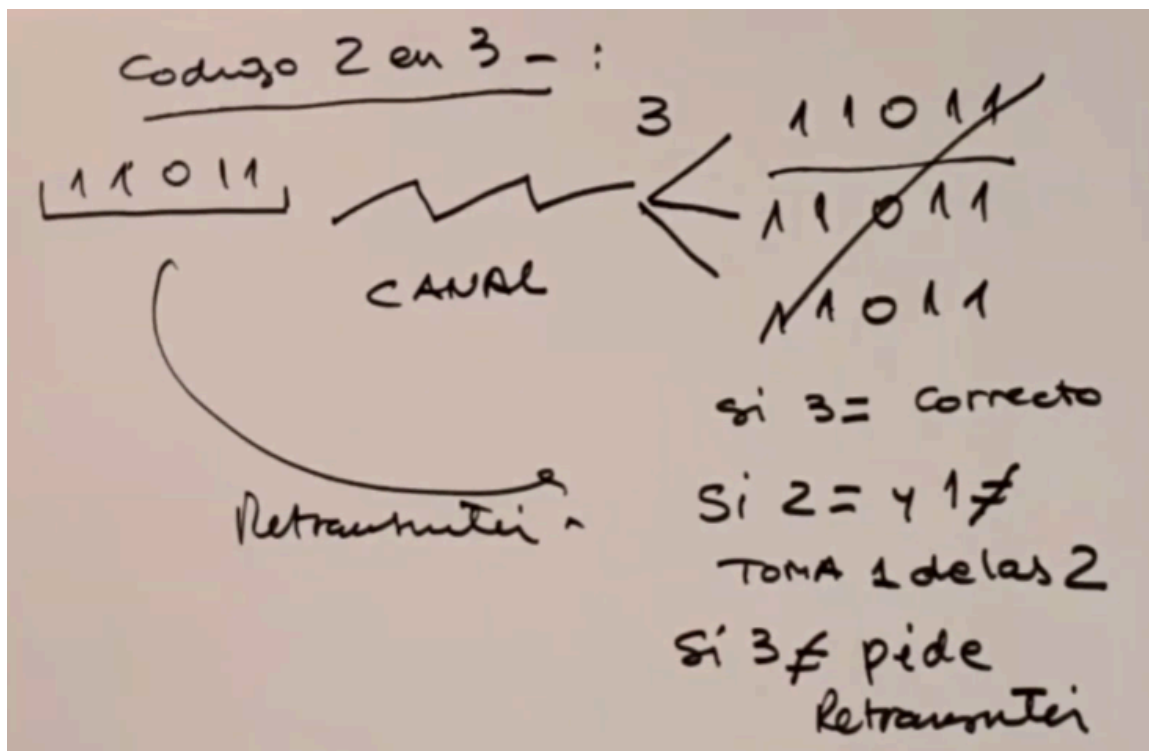
¿Qué son los Códigos Redundantes?

Los códigos redundantes son métodos que agregan información extra (llamada redundancia) a los datos originales con el objetivo de detectar y, en algunos casos, corregir errores durante la transmisión o almacenamiento de información. Esta redundancia no forma parte de los datos que se desean transmitir, sino que se utiliza como mecanismo de verificación y protección contra fallos.

Código 2 en 3

¿Qué es el Código 2 en 3?

El código 2-en-3 significa que mandás cada bloque de datos tres veces. Si las tres transmisiones son iguales, está todo bien. Si dos son iguales y una diferente, nos quedamos con una de las iguales. Y si las tres son diferentes, no podemos adivinar, así que pedimos una retransmisión. Es una forma de asegurarte de que los datos lleguen bien incluso si hay ruido.



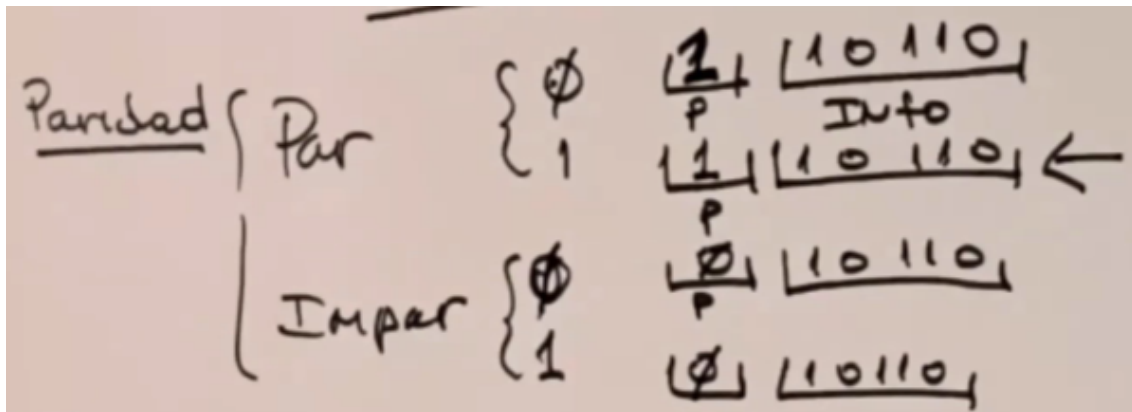
En si, nos sirve para detectar y corregir una cadena con errores en sus bits.

Paridad

¿Qué es la Paridad?

La paridad es un método simple de detección de errores que se utiliza en la transmisión de datos para verificar si la información llegó correctamente. La idea básica es agregar un bit extra (llamado bit de paridad) al final de un bloque de datos.

Los tipos de paridad son:



Al transmitir, el transmisor genera el bloque de datos y calcula el bit de paridad según el esquema (par o impar).

El receptor recibe el bloque de datos (incluyendo el bit de paridad), y calcula si la cantidad total de bits "1" cumple con la regla de paridad:

- Si cumple, se considera que el bloque llegó correctamente.
- Si no cumple, se detecta un error y se puede pedir una retransmisión.

En si, se utiliza para detectar y corregir UN bit erróneo en una cadena binaria.

¿A que hace referencia la Paridad Vertical simple o a Nivel Carácter?

La paridad vertical simple se aplica a cada bloque individual de datos, como un octeto (8 bits). Consiste en agregar un bit de paridad al final del bloque para asegurar que el número total de bits "1" cumpla con la regla de paridad par o impar, según lo definido.

00110010 1 se agrega un uno que permita obtener paridad par,
01001011 0 se agrega un cero para lograr paridad par.

Paridad Entrelazada

¿Qué es la Paridad Horizontal y Vertical (Codigo Entrelazado)?

Es un método que permite transmitir bloques de caracteres ASCII de 8 bits, acompañados del conjunto de bits de paridad vertical y horizontal que permita la detección y corrección de unos o mas bits, siempre y cuando se encuentre en la misma fila y/o columna.

I_0	I_1	I_2	I_3	PH	
0	1	0	1	0	
0	1	1	1	1	
1	1	0	1	1	
1	0	1	1	1	
1	1	1	0	1	
1	0	1	1	1	
0	1	1	0	0	
PV	0	1	1	1	1

I_0	I_1	I_2	I_3	PH	
0	1	0	1	0	
0	0	1	1	1	
1	1	0	1	1	
1	0	1	1	1	
1	1	1	0	1	
1	0	1	1	1	
0	1	1	0	0	
PV	0	1	1	1	1

En la primer tabla, en cada fila, contamos los bits "1". Si el total es impar, agregamos un 1 en el bit de paridad horizontal para hacerlo par. Si ya es par, agregamos un 0.

Ahora, para cada columna (incluyendo los bits de datos y los de paridad horizontal), contamos los bits "1". Si el total es impar, agregamos un 1 en el bit de paridad vertical para hacerlo par. Si ya es par, agregamos un 0.

En la segunda tabla, al analizar las paridades, se identifica, que la fila 2 (I_1) es conflictiva por su paridad horizontal incorrecta, y la columna 1 también conflictiva por su paridad vertical incorrecta.

El cruce de la fila conflictiva y la columna conflictiva indica la posición exacta del bit erróneo. En la posición donde se cruzan fila y columna conflictivas, el bit se invierte, si era 0 se cambia a 1, o viceversa.

¿Cuáles son las ventajas y desventajas de la Paridad Horizontal y Vertical?

Las ventajas son:

- Permite identificar la posición exacta de un error en un único bit y corregirlo de forma automática.
- Puede detectar y corregir múltiples errores en las diferentes filas o columnas.

Las desventajas:

- Si ocurren múltiples errores en una misma fila o columna, el método puede detectar que hay un problema, pero no puede identificar ni corregir con precisión los bits afectados.
- Para cada fila y columna de datos se deben agregar bits de paridad, lo que aumenta el tamaño total de los datos a transmitir. Esto reduce la eficiencia en términos de ancho de banda.
- Si el volumen de datos es grande, utiliza muchos ciclos de CPU, ya que, requiere realizar numerosos cálculos lógicos y recorridos sobre los datos para calcular, verificar y corregir paridades, lo que incrementa significativamente el tiempo de procesamiento.

Código de Hamming

¿Qué es un Código Binario?

Un código binario es una forma de representar información usando únicamente los símbolos 0 y 1. Cada combinación de ceros y unos (bits) representa algo: un número, una letra, un comando, etc.

¿A que hace referencia la Distancia entre dos combinaciones Binarias?

La distancia entre dos combinaciones binarias se refiere al número de bits que deben cambiarse en una de las combinaciones para convertirla en la otra. Este concepto se mide usando algo llamado distancia de Hamming.

Se realiza de la siguiente manera:

- Se toma dos combinaciones binarias.
- Se compara bit por bit.
- Se cuenta cuántos bits son diferentes.

Por ejemplo, en las combinaciones 1001 y 1100, la distancia entre ambas es igual a 2.

¿Qué es la Distancia Mínima de un código?

La distancia mínima de un código es el número mínimo de bits que deben cambiarse para convertir cualquier combinación válida de ese código en otra combinación válida.

Por ejemplo, supongamos que tenemos tres caritas diferentes, que representan combinaciones válidas de un código. Les doy números binarios para identificarlas:

Carita A: 😊 → 000 (sin cejas ni bigote).
Carita B: 😎 → 101 (tiene lentes oscuros y bigote).
Carita C: 😏 → 111 (tiene lentes y cejas).

La distancia entre dos caritas es cuántos detalles tenemos que cambiar para que una carita se convierta en otra.

De Carita A (😊 : 000) a Carita B (😎 : 101):

- Cambiás dos detalles:
 - Agregás lentes (cambio el 0 por un 1).
 - Agregás bigote (cambio el 0 por un 1).
- Distancia = 2.

De Carita A (😊 : 000) a Carita C (😏 : 111):

- Cambiás tres detalles:
 - Agregás lentes (cambio el 0 por un 1).
 - Agregás cejas (cambio el 0 por un 1).
 - Agregás bigote (cambio el 0 por un 1).
- Distancia = 3.

De Carita B (😎 : 101) a Carita C (😏 : 111):

- Cambiás un detalle:
 - Agregás cejas (cambio el 0 por un 1).
- Distancia = 1.

La distancia mínima del código es 1, porque es el menor número de cambios necesarios para transformar una carita válida en otra.

En síntesis, la distancia mínima es como decir qué tan diferentes son las caritas. Si tenemos que cambiar 1 solo detalle, están demasiado cerca y es difícil saber si hubo un error. Pero si tenemos que cambiar 3 detalles, las diferencias son claras y podemos detectar y corregir errores con confianza.

¿Qué es la Distancia de Hamming?

La distancia de Hamming es una medida que indica cuántos bits diferentes hay entre las combinaciones válidas de un código. Esta distancia mínima determina

cuántos errores se pueden detectar o corregir.

Distancia de Hamming

$D_m = 2X + 1$

$D_m = 2 * 0 + 1 = 1$
Distancia HAMMING = 1

Mensaje	
0	Válido
1	Válido

Se debe modificar 1 bit

$D_m = 2 * 1 + 1 = 3$
Distancia HAMMING = 2

Mensaje		
0	0	Válido
0	1	Invalido
1	0	Invalido
1	1	Válido

Se pueden detectar errores de 1 bit
(Caso bit Paridad)

$D_m = 2 * 3 + 1 = 7$
Distancia HAMMING = 3

Mensaje			
0	0	0	Válido
0	0	1	Invalido
0	1	0	Invalido
0	1	1	Invalido
1	0	0	Invalido
1	0	1	Invalido
1	1	0	Invalido
1	1	1	Válido

Detecta y corrige errores de 1 bit

El resultado de la operación que se realiza con la fórmula de la distancia de Hamming ($D_m=2X+1$) representa la distancia mínima entre combinaciones válidas de un código.

Los casos anteriores pueden explicarse como:

- Caso 1: Distancia mínima = 1
Si dos combinaciones válidas están separadas por 1 bit, significa que son muy similares, y un error podría hacer que una combinación válida se convierta en otra:
 - Combinaciones válidas: 0, 1.
 - Distancia entre ellas: 1 bit.
 - Problema: Si ocurre un error, no puedes saber si el dato original era un 0 o un 1.
- Caso 2: Distancia mínima = 2
Si dos combinaciones válidas están separadas por 2 bits, significa que hay

más separación entre ellas:

- Combinaciones válidas: 00, 11.
 - Distancia entre ellas: 2 bits.
 - Ventaja: Si cambia 1 bit, sabrás que ocurrió un error porque el dato resultante no será válido (ejemplo: 01 o 10 no están permitidos).
 - Problema: No puedes corregir el error porque no sabes cuál era el dato original (00 o 11).
- Caso 3: Distancia mínima = 3
Si dos combinaciones válidas están separadas por 3 bits, la distancia es suficiente para detectar y corregir un error:
 - Combinaciones válidas: 000, 111.
 - Distancia entre ellas: 3 bits.
 - Ventaja: Si cambia 1 bit (ejemplo: 001), sabrás que el dato más cercano es 000, por lo que puedes corregirlo.

¿Qué es el Código de Hamming?

El Código de Hamming es un método de corrección de errores que permite detectar y corregir errores de un solo bit en un mensaje durante la transmisión de datos. Este código utiliza una combinación de bits de información y bits de paridad para garantizar que los errores simples puedan ser corregidos automáticamente, mejorando la confiabilidad de las transmisiones.

El proceso que se lleva a cabo para la codificación con Hamming es:

1. Relación entre los bits de información y los bits de paridad

La fórmula que define el Código de Hamming es: $2^p \geq i + p + 1$ Donde:

- p : Cantidad de bits de paridad que se necesitan.
- i : Cantidad de bits de información a transmitir.

2. El mensaje final incluye tanto los bits de información (i) como los bits de paridad (p).

Los bits de paridad se colocan en las posiciones que son potencias de 2 dentro del mensaje (1, 2, 4, 8, etc.).

Ejemplo para 7 posiciones totales:

Posición 1 → p0.

Posición 2 → p1.

Posición 4 → p2.

Las posiciones restantes (3, 5, 6, 7) se llenan con los bits de información.

Cada bit de paridad verifica un subconjunto específico de posiciones:

p0 (posición 1) verifica las posiciones: 1, 3, 5, 7.

p1 (posición 2) verifica las posiciones: 2, 3, 6, 7.

p2 (posición 4) verifica las posiciones: 4, 5, 6, 7.

3. Control y corrección

EJEMPLO

$c_0 = 1 \oplus 0 \oplus 1 \oplus 1 =$	1
$c_1 = 1 \oplus 1 \oplus 0 \oplus 0 =$	0
$c_2 = 1 \oplus 1 \oplus 1 \oplus 1 =$	0

Posición Error: 001

Si el bit de control es igual a 0 es correcto, si es 1 es incorrecto. La posición del error se determina según los valores obtenidos en los bits de control ordenados en forma decreciente, es decir c2, c1 y c0. Si todos los bits de control son igual a cero no se detecta error en el mensaje.

¿Cómo es el flujo de transmisión cuando se usa Hamming?

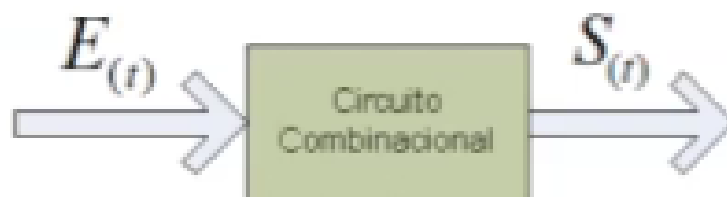
El flujo es el siguiente:



Circuitos Combinacionales

¿Qué son los Circuitos Combinacionales?

Un circuito combinatorial es un sistema en el que la salida ($S(t)$) depende únicamente de las entradas ($E(t)$) en el mismo instante de tiempo t . Es decir, no tiene "memoria" ni guarda información sobre estados pasados. La salida se genera únicamente a partir de las combinaciones actuales de las entradas.



El funcionamiento de los circuitos combinatoriales se basa en el uso de puertas lógicas (como AND, OR, NOT, XOR, etc.) que combinan las entradas de acuerdo con reglas predefinidas (álgebra de Boole). Estas puertas lógicas procesan las entradas y generan las salidas.

Los valores de salida no se retroalimentan hacia el circuito. Esto significa que no hay "feedback" como en los circuitos secuenciales.

¿Qué son las Formas Normales?

Las formas normales son maneras de organizar fórmulas lógicas de manera estándar para facilitar su análisis y manipulación. En lógica proposicional, se usan dos tipos principales:

- Forma Normal Conjuntiva (FNC): Es una conjunción (operación AND: "y") de varias disyunciones (operaciones OR: "o").

Se ve así:

$$(L11 \vee L12 \vee \dots \vee L1n) \wedge (L21 \vee L22 \vee \dots \vee L2n) \wedge \dots \wedge (Lm1 \vee Lm2 \vee \dots \vee Lmn)$$

Forma Normal Conjuntiva (FNC) Se obtiene a partir de su tabla de verdad, en las filas donde la función vale cero, se forman las sumas de todas las variables reemplazando los unos de cada combinación de valores por sus respectivas variables negadas, y los ceros por sus correspondientes variables sin negar. Luego, se realiza el producto de los maxitérminos así obtenidos

A	B	C	Maxitérminos	S
0	0	0	$A+B+C$	1
0	0	1	$A+B+\bar{C}$	0
0	1	0	$A+\bar{B}+C$	0
0	1	1	$A+\bar{B}+\bar{C}$	1
1	0	0	$\bar{A}+B+C$	0
1	0	1	$\bar{A}+B+\bar{C}$	1
1	1	0	$\bar{A}+\bar{B}+C$	1
1	1	1	$\bar{A}+\bar{B}+\bar{C}$	0

$$F = (A+B+\bar{C})(A+\bar{B}+C)(\bar{A}+B+C)(\bar{A}+\bar{B}+\bar{C})$$

- Forma Normal Disyuntiva (FND): Es una disyunción (operación OR: "o") de varias conjunciones (operaciones AND: "y").

Se ve así:

$$(L11 \wedge L12 \wedge \dots \wedge L1n) \vee (L21 \wedge L22 \wedge \dots \wedge L2n) \vee \dots \vee (Lm1 \wedge Lm2 \wedge \dots \wedge Lmn)$$

En ambas formas Lij son literales, que pueden ser una variable (como A) o su negación ($\neg A$).

Forma Normal Disyuntiva (FND) Se obtiene a partir de la tabla de verdad, en las filas donde la función vale 1, tomando el producto de todas las variables, reemplazando los ceros por sus respectivas negadas y los 1 por sus correspondientes variables sin negar. Luego se realiza la suma de los minitérminos así determinados.

Ejemplo: supongamos que la siguiente tabla corresponde al funcionamiento de un circuito cuya salida es 1 cuando sus entradas tienen un número par de unos, o ninguna entrada es uno

A	B	C	Minitérminos	S
0	0	0	$\bar{A} \cdot \bar{B} \cdot \bar{C}$	1
0	0	1	$\bar{A} \cdot \bar{B} \cdot C$	0
0	1	0	$\bar{A} \cdot B \cdot \bar{C}$	0
0	1	1	$\bar{A} \cdot B \cdot C$	1
1	0	0	$A \cdot \bar{B} \cdot \bar{C}$	0
1	0	1	$A \cdot \bar{B} \cdot C$	1
1	1	0	$A \cdot B \cdot \bar{C}$	1
1	1	1	$A \cdot B \cdot C$	0

$$FND = \bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot B \cdot C + A \cdot \bar{B} \cdot C + A \cdot B \cdot \bar{C}$$

¿Qué son las Maxitérminos y Minitérminos?

- **Minitérminos:** Un minitérmino es un producto lógico (AND) de todas las variables de una función lógica, donde cada variable aparece en su forma directa o negada, dependiendo del caso. Se usan para construir la suma de productos (forma normal conjuntiva).

Los minitérminos se combinan en una suma de productos (AND de variables combinado con OR).

$$F(A, B) = (\bar{A} \cdot \bar{B}) + (\bar{A} \cdot B) + (A \cdot \bar{B}) + (A \cdot B)$$

- **Maxitérminos:** Un maxitérmino es una suma lógica (OR) de todas las variables de una función lógica, donde cada variable aparece en su forma directa o negada, dependiendo del caso.

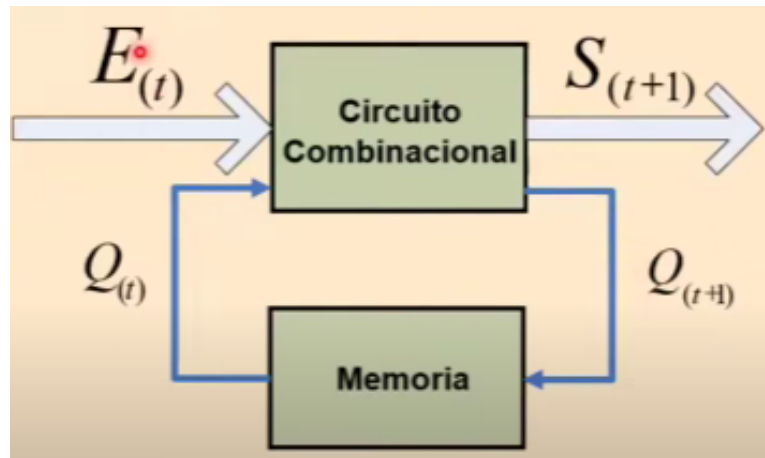
Los maxitérminos se combinan en un producto de sumas (OR de variables combinado con AND).

$$F(A, B) = (A + B) \cdot (A + \bar{B}) \cdot (\bar{A} + B) \cdot (\bar{A} + \bar{B})$$

Circuitos Secuenciales

¿Qué son los Circuitos Secuenciales?

Los circuitos secuenciales son un tipo de circuito lógico donde las salidas dependen no solo de las entradas actuales, sino también del estado previo del sistema. Esto significa que tienen memoria, lo que les permite almacenar información sobre lo que sucedió antes y usarla para decidir qué hacer después. A diferencia de los circuitos combinacionales, que simplemente procesan las entradas y generan una salida inmediata, los circuitos secuenciales consideran tanto las entradas como los estados almacenados en su memoria.



¿Qué funciones existen en los Circuitos Secuenciales?

Las funciones que existen son las siguientes:

- Función de transición de estado: La función de transición de estado determina cuál será el próximo estado del sistema (Q_{t+1}) en función del estado actual (Q_t) y las entradas actuales (E_t). En el diagrama, esta función está representada como:

$$Q_{t+1} = G(Q_t, E_t)$$

Términos:

- Q_{t+1} : Es el **estado del sistema en el siguiente instante de tiempo**. Es el valor que será almacenado en la memoria al final del ciclo actual.
 - G : Es el circuito lógico (circuito combinacional) que calcula el próximo estado.
 - Q_t : Es el **estado actual**, es decir, el valor que la memoria está almacenando en este momento.
 - E_t : Representa las **entradas externas** en el instante actual.
- **Función de salida:** La función de salida describe cómo el sistema genera la salida (S_{t+1}) en función del estado actual (Q_t) y las entradas actuales (E_t). En el diagrama, está representada como:

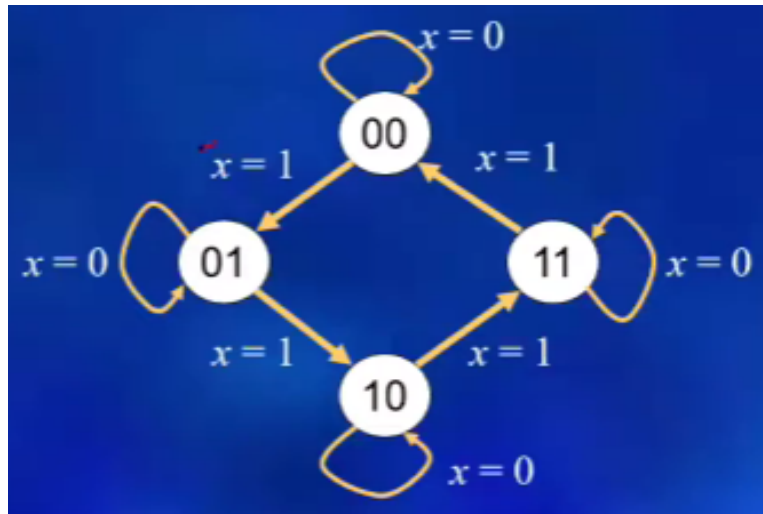
$$S_{t+1} = F(Q_t, E_t)$$

Términos:

- S_{t+1} : Es la **salida del sistema en el siguiente instante**. Es lo que el circuito entrega al exterior como resultado.
- F : Es la lógica que determina la salida (dentro del circuito combinacional).
- Q_t : Es el estado actual del sistema, que contribuye a calcular la salida.
- E_t : Representa las entradas actuales, que también influyen en la salida.

¿Qué es un Diagrama de Transición?

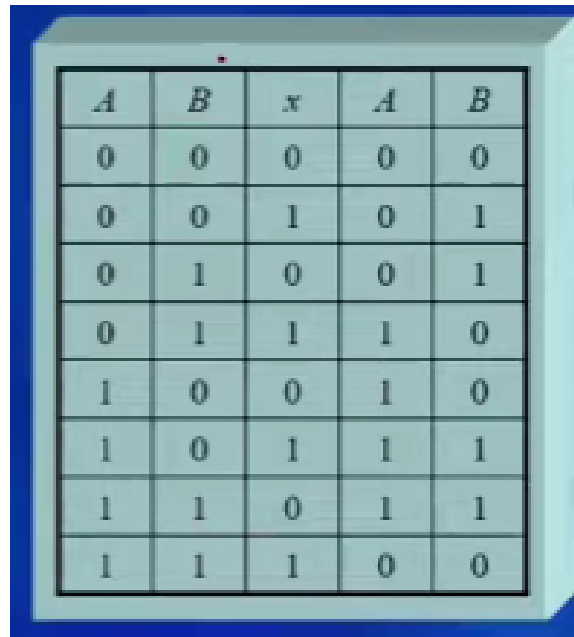
Un diagrama de transición es una representación gráfica de los estados de una máquina y las transiciones posibles entre ellos. En este diagrama:



- Cada círculo representa un estado.
- Cada flecha representa una transición entre estados.
- Las etiquetas en las flechas indican las condiciones que deben cumplirse (generalmente las entradas) para que ocurra la transición.
- Opcionalmente, las salidas del sistema pueden indicarse en los estados o en las transiciones.

¿Qué es un Tabla de Estados?

La tabla de estados es la versión tabular de lo que muestra el diagrama de transición. Resume:



<i>A</i>	<i>B</i>	<i>x</i>	<i>A</i>	<i>B</i>
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	1	0
1	0	1	1	1
1	1	0	1	1
1	1	1	0	0

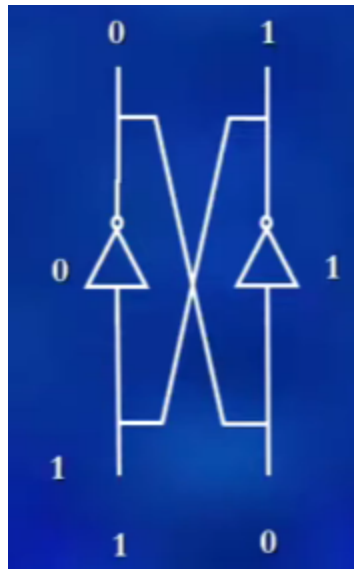
- Los estados actuales.
- Las entradas posibles.
- El próximo estado al que transita el sistema en función de esas entradas.
- Las salidas generadas, si las hay.

Biestables

¿Qué es un Biestable?

Un biestable es un circuito lógico secuencial que tiene la capacidad de almacenar un bit de información y mantenerlo en uno de dos estados estables: 0 o 1. Es una pieza fundamental en la electrónica digital porque sirve como base para memorias, registros y contadores. Su característica principal es que tiene memoria, lo que significa que puede "recordar" el estado en el que se encuentra hasta que una señal externa lo haga cambiar. Este circuito funciona mediante realimentación, es decir, la salida del biestable se conecta como entrada, lo que permite que mantenga su estado de forma estable.

Un biestable básico puede representarse conceptualmente con dos compuertas NOT conectadas entre sí, formando un bucle de realimentación.



Supongamos que inicialmente, la salida de la primera compuerta NOT es 0.

Esta salida se conecta a la entrada de la segunda compuerta NOT, que la invierte y produce una salida de 1.

La salida de la segunda compuerta (1) se realimenta como entrada de la primera compuerta NOT, que la invierte de nuevo y genera 0.

Este ciclo de retroalimentación asegura que el circuito mantenga el estado de 0 y 1 en sus salidas respectivas.

Si una señal externa interrumpe este equilibrio (por ejemplo, forzando temporalmente una de las salidas a 1), el circuito ajustará sus valores para reflejar este cambio.

¿Qué es una tabla de Funcionamiento?

En la tabla de funcionamiento, el enfoque está en cómo las entradas afectan el estado actual (Q_t) y determinan el próximo estado (Q_{t+1}). Aquí, los estados (Q_t y Q_{t+1}) son salidas del sistema porque están determinados por las combinaciones del Biestable con el que estemos trabajando.

¿Qué es una Tabla de Excitación?

En la tabla de excitación, el enfoque es determinar qué entradas se necesitan para hacer que el biestable pase de un estado actual (Q_t) a un próximo estado deseado (Q_{t+1}). Aquí, los estados (Q_t y Q_{t+1}) son entradas, ya que son condiciones dadas,

y las señales de control (del Biestable con el que trabajemos) son las salidas que debemos encontrar.

¿Qué es el Clock?

El clock, o reloj, es una señal periódica que actúa como el corazón de los circuitos sincrónicos, marcando el ritmo al que todos los componentes del sistema deben operar. Esta señal oscila constantemente entre dos niveles lógicos, bajo (0) y alto (1), y define en qué momento los elementos del circuito deben actualizar su estado. En esencia, es como un metrónomo que establece un compás preciso, asegurando que cada parte del sistema digital funcione de manera ordenada y sincronizada.

En los circuitos sincrónicos, como los flip-flops, el clock juega un papel fundamental. Aunque las entradas al circuito puedan cambiar en cualquier momento, el flip-flop no tomará en cuenta esos cambios ni actualizará su estado hasta que el clock indique que es el momento adecuado. Este momento específico ocurre durante una transición del reloj, que puede ser el paso de bajo a alto (borde de subida) o de alto a bajo (borde de bajada). Así, el reloj garantiza que las operaciones dentro del circuito se realicen en momentos definidos y que todos los componentes trabajen siguiendo un mismo "ritmo".

Sin esta señal de reloj, los circuitos digitales podrían operar de manera desordenada, procesando entradas y actualizando salidas en momentos impredecibles. Esto podría generar errores, ya que diferentes componentes podrían desincronizarse, causando resultados inesperados. El clock evita esto al establecer un ciclo repetitivo y predecible en el que todo el sistema se coordina.

En conclusión, el clock es una señal que regula y organiza el trabajo de los circuitos digitales. Es como un director de orquesta que asegura que cada instrumento entre en el momento justo, evitando que el sistema pierda el ritmo.

¿Qué es el tiempo de Estabilización?

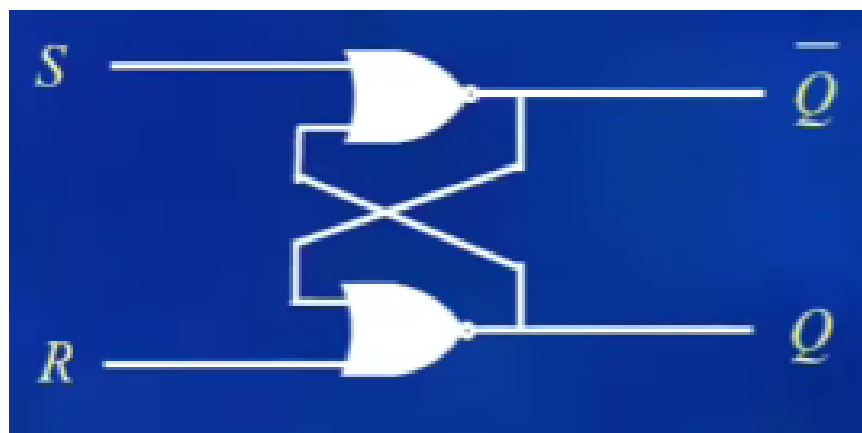
El tiempo de estabilización en un flip-flop es el tiempo que el circuito necesita para que sus salidas alcancen un estado confiable y estable después de un cambio en las entradas. Este tiempo está influenciado por el retraso en la propagación de las señales a través de las compuertas lógicas internas y la realimentación del circuito. En sistemas sincrónicos, este tiempo se mide en ciclos

del reloj, ya que el flip-flop sólo actualiza su estado en momentos definidos por el pulso del clock.

¿Qué tipos de Biestables existen?

Los Biestables existentes son:

- Flip-flop RS Asincrónico: El flip-flop RS es un circuito que utiliza compuertas lógicas para implementar la funcionalidad de "Set" (poner la salida en 1) y "Reset" (poner la salida en 0). Es capaz de mantener su estado (ya sea 0 o 1) indefinidamente hasta que una señal externa lo modifique.



- **Tiene dos entradas:**
 - S (Set): Ordena que la salida (Q) pase a 1.
 - R (Reset): Ordena que la salida (Q) pase a 0.
- **Dos salidas:**
 - Q: Es la salida principal que representa el estado del flip-flop.
 - $\bar{Q}$: Es la salida complementaria de Q (si $Q = 1$, entonces $\bar{Q} = 0$, y viceversa).
- **Y un estado inicial.**

Tabla de Comportamiento:

Tabla de Excitación:

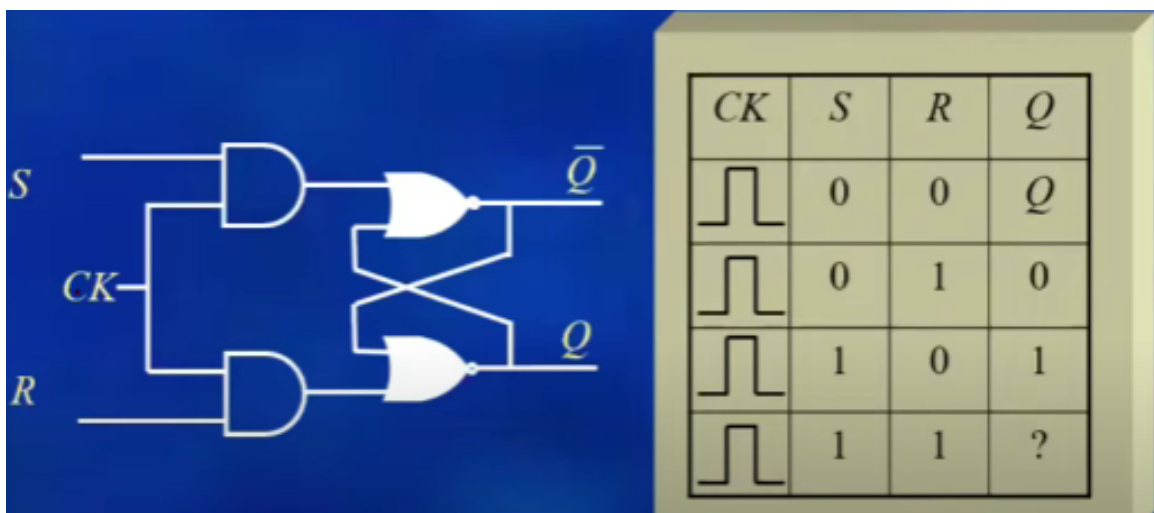
S	R	Q(t)	Q(t+1)
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	?
1	1	1	?

Q(t)	Q(t+1)	S	R
0	0	0	X
0	1	1	0
1	0	0	1
1	1	X	0

Representación en bloque:

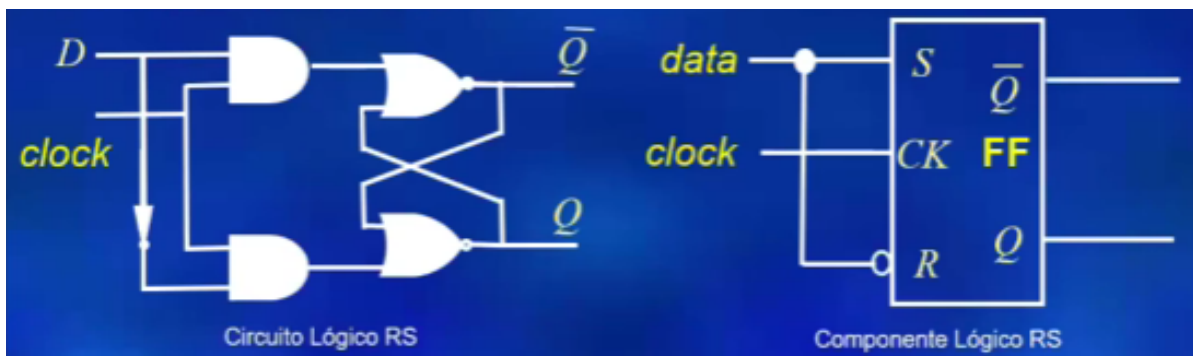


- Flip-flop RS Sincrónico: Un flip-flop sincrónico es un tipo de circuito secuencial que actualiza su estado (las salidas Q y $\bar{Q}$) sólo en momentos específicos definidos por una señal de clock (reloj). Esto significa que el flip-flop no responde a cambios en sus entradas (S y R, en este caso) de manera inmediata, sino que espera al instante exacto marcado por el pulso del reloj para procesar las señales y actualizar sus salidas.

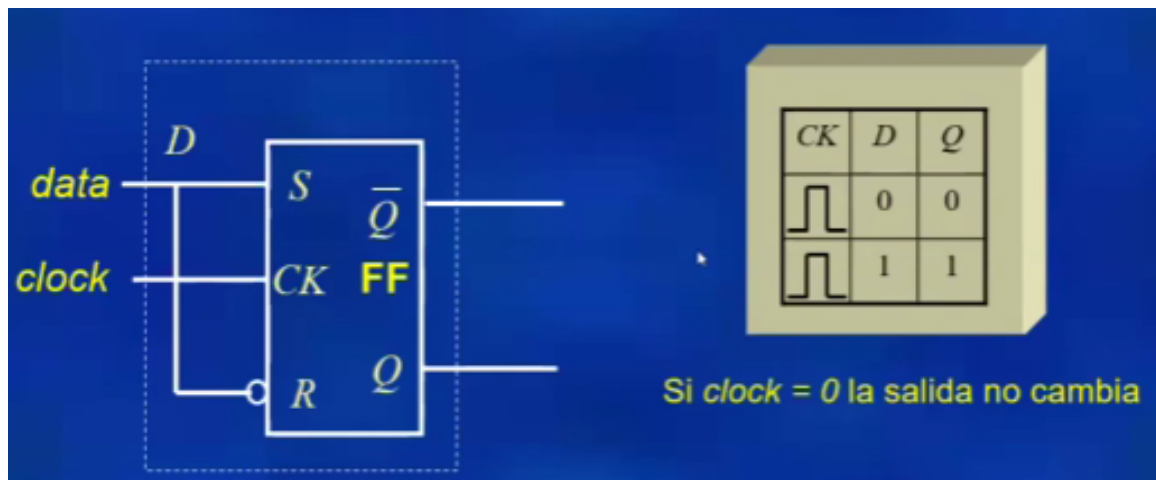




- Flip-flop D Síncrono: El flip-flop D (o flip-flop tipo "Data") es una variante del flip-flop RS diseñada para evitar el problema de indeterminación que ocurre en el flip-flop RS cuando ambas entradas (S y R) son iguales a 1. En lugar de manejar dos entradas separadas (S y R), el flip-flop D utiliza una sola entrada llamada D (de "data" o dato), que simplifica su uso al garantizar siempre un comportamiento definido.



El flip-flop D tiene una estructura interna basada en el flip-flop RS, pero con un diseño modificado para evitar los problemas de indeterminación. La entrada D (Data) se conecta directamente a la entrada S (Set) del flip-flop RS. A su vez, la entrada R (Reset) del flip-flop RS se obtiene al aplicar una compuerta lógica NOT a D, lo que asegura que S y R nunca sean iguales a 1 al mismo tiempo. Este diseño simplifica el comportamiento del flip-flop, ya que la entrada D controla de forma directa el valor lógico que se almacena en Q (0 o 1).



En este Flip-flop, el clock es esencial para controlar y sincronizar el funcionamiento del circuito. Sin el clock, el flip-flop respondería directamente a los cambios en la entrada D. La ausencia de una señal en D se interpreta como un cero lógico ($D=0$), lo que activa automáticamente la condición de reset ($R=1$), manteniendo la salida $Q=0$. Esto significa que, sin un clock, el circuito quedaría en un estado de reset continuo mientras $D=0$, perdiendo la capacidad de almacenar datos de manera confiable.

Tabla de Comportamiento:

D	Q_t	Q_{t+1}
0	0	0
0	1	0
1	0	1
1	1	1

Tabla de Excitación:

Q_t	Q_{t+1}	D
0	0	0
0	1	1
1	0	0
1	1	1

- Flip-flop JK:** El flip-flop JK es una mejora del flip-flop RS diseñada para evitar el problema de indeterminación que ocurre cuando ambas entradas S y R son iguales a 1. En el flip-flop JK, las entradas se llaman J (Set) y K (Reset). Cuando $J=1$ y $K=0$, el flip-flop pone la salida $Q=1$ (estado Set). Si $J=0$ y $K=1$, el flip-flop pone $Q=0$ (estado Reset). Cuando ambas entradas son 0, el flip-flop mantiene su estado actual, sin cambios. La funcionalidad clave del flip-flop JK ocurre cuando $J=K=1$: en este caso, el circuito complementa la salida, cambiando Q de 1 a 0 o de 0 a 1, dependiendo de su estado anterior.

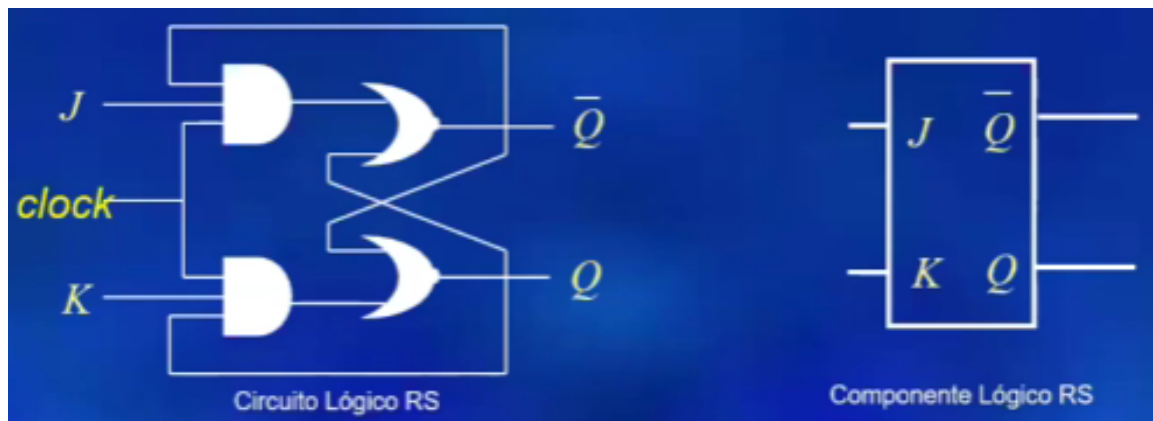


Tabla de Comportamiento: Tabla de Excitación:

J	K	Q_t	Q_{t+1}
0	0	0	0
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Q_t	Q_{t+1}	J	K
0	0	0	x
0	1	1	x
1	0	x	1
1	1	x	0

- Flip-flop T: El flip-flop T (Toggle o Conmutación) es una variante del flip-flop JK que simplifica su funcionalidad al reducir las entradas a una sola llamada T. Este flip-flop está diseñado específicamente para alternar entre los estados $Q=0$ y $Q=1$ cada vez que la entrada T está activada ($T=1$) y ocurre un pulso de reloj. En cambio, si $T=0$, el flip-flop mantiene el último estado almacenado, es decir, Q no cambia.

Es decir, simplifica el cambio de estado del flip-flop JK cuando $J=K=1$, reemplazándolo con un $T=1$ y un pulso de reloj, el flip-flop T ya alterna entre Q y $\bar{Q}$ automáticamente, haciendo su diseño más simple para realizar toggles.

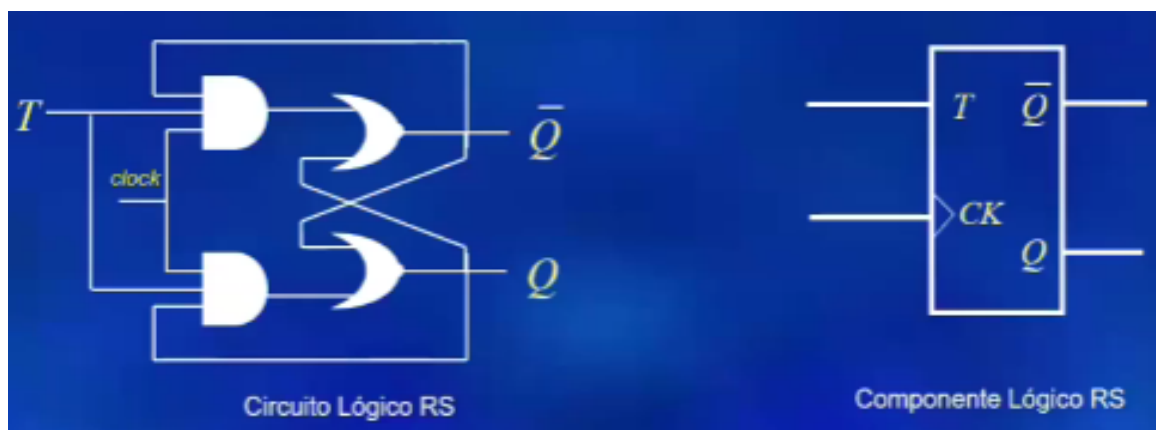


Tabla de Comportamiento:

T	Q_t	Q_{t+1}
0	0	0
0	1	1
1	0	1
1	1	0

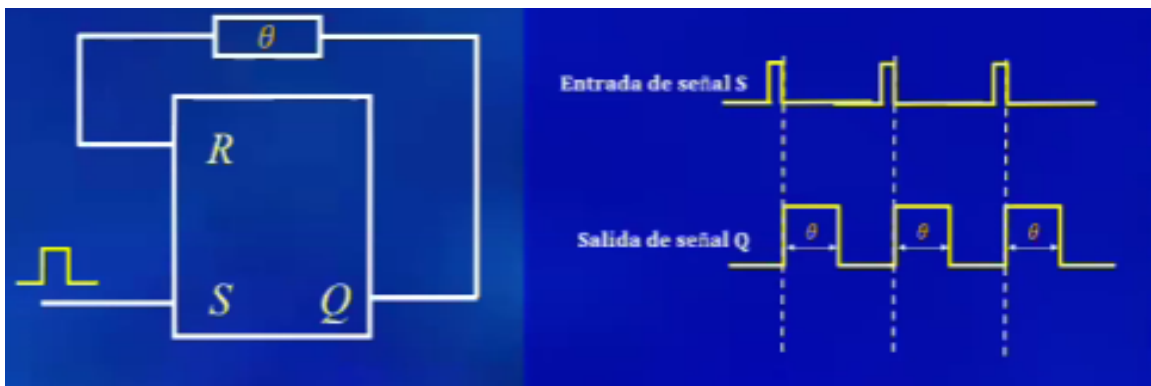
Tabla de Excitación:

Q_t	Q_{t+1}	T
0	0	0
0	1	1
1	0	1
1	1	0

Monoestable

¿Qué es un Monoestable?

Un monoestable es un tipo de circuito lógico secuencial que tiene dos estados posibles, pero solo uno de ellos es estable. Esto significa que el circuito, en su estado natural, se encuentra en un estado de reposo o equilibrio (el estado estable) y únicamente cambia temporalmente al otro estado (estado inestable) cuando recibe un impulso de entrada. Después de un tiempo determinado, el circuito vuelve automáticamente a su estado estable, independientemente de si la entrada sigue activa o no.



¿Cómo funciona un Monoestable?

Su funcionamiento es el siguiente:

1. **Estado inicial y estable:**

- El circuito está diseñado para permanecer en un estado estable, que generalmente es 0 (estado de reposo).
- Sin la influencia de una señal externa, la salida (Q) del monoestable permanece fija en este estado estable ($Q=0$).

2. **Cambio por impulso de entrada:**

- Cuando se aplica un pulso de entrada (S), el circuito cambia su estado de salida a $Q=1$ (estado inestable).
- Este cambio es temporal y depende de un temporizador interno (θ), que controla cuánto tiempo el circuito permanece en este estado.

3. **Regreso al estado estable:**

- Una vez que el tiempo definido por el temporizador (Θ) se cumple, el monoestable genera un RESET automático, haciendo que la salida regrese al estado estable ($Q=0$).
- Incluso si el pulso de entrada S sigue activo, el circuito no mantiene el estado inestable por más tiempo que el definido por el temporizador.

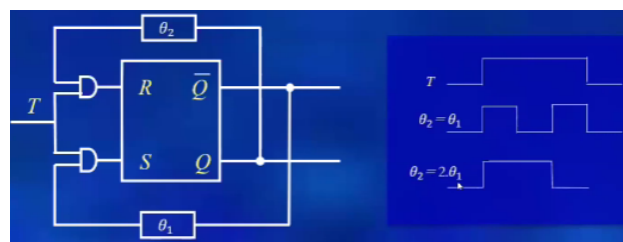
4. **Salida temporizada:**

- La duración del estado inestable ($Q=1$) está determinada por el temporizador (Θ), el cual puede ajustarse utilizando elementos como resistencias y capacitores.

Bascula

¿Qué es la Bascula?

Una báscula, también conocida como circuito astable, es un sistema lógico que no tiene ningún estado estable. Esto significa que, a diferencia de los circuitos monoestables o biestables, la báscula cambia continuamente entre sus dos estados posibles ($Q=0$ y $Q=1$) de manera automática y periódica. Este comportamiento oscilatorio se produce sin necesidad de una entrada adicional después de su configuración inicial, lo que convierte a la báscula en un generador natural de señales de onda cuadrada.



¿Cómo funciona una Bascula?

1. **Estados inestables:**

- La báscula tiene dos posibles estados: $Q=0$ (apagado) y $Q=1$ (encendido). Sin embargo, ninguno de estos es estable, lo que significa que el circuito no permanece en un estado indefinidamente. Una vez que alcanza un estado, cambia automáticamente al otro después de un tiempo definido por los temporizadores.

2. **Temporizadores ($\Theta 1$ y $\Theta 2$):**

- Cada estado está asociado a un temporizador:
 - $\Theta 1$ define cuánto tiempo el circuito permanece en $Q=1$.
 - $\Theta 2$ define cuánto tiempo el circuito permanece en $Q=0$.
- Por ejemplo:
 - Si $\Theta 1$ representa 1 segundo y $\Theta 2$ representa 2 segundos ($\Theta 2=2\cdot\Theta 1$), el circuito estará en $Q=1$ durante 1 segundo y luego cambiará a $Q=0$ durante 2 segundos antes de volver a $Q=1$.

3. **Oscilación automática:**

- La báscula cambia continuamente entre $Q=1$ y $Q=0$, generando una señal periódica. La duración de los estados depende de los valores de $\Theta 1$ y $\Theta 2$. Si ambos tiempos son iguales ($\Theta 1=\Theta 2$), la señal es simétrica (ciclo de trabajo del 50 %). Si $\Theta 2$ es dos veces $\Theta 1$, la señal será asimétrica: Q estará en 1 durante un tercio del tiempo total y en 0 durante los dos tercios restantes.

4. **Relación con la onda cuadrada:**

- La salida de la báscula (Q) tiene la forma de una onda cuadrada. Su período total es la suma de $\Theta 1+\Theta 2$, y el ciclo de trabajo (porcentaje del tiempo que $Q=1$) depende de la relación entre los temporizadores:
 - Ciclo de trabajo = $(\Theta 1/(\Theta 2+\Theta 1))\cdot 100$.
- Si $\Theta 1=1$ segundo y $\Theta 2=2$ segundos, el ciclo de trabajo será $(1/(1+2))\cdot 100=33.33\%$.

5. **Implementación técnica:**

- Se puede construir una báscula conectando las dos entradas de un flip-flop JK ($J=K=1$). Esto asegura que el circuito alterne continuamente entre sus dos estados en cada pulso. También puede implementarse con un temporizador integrado, como el famoso 555, ajustando los tiempos mediante resistencias y capacitores.

Registros

¿Qué son los Registros?

Un registro es un conjunto de flip-flops organizados de manera lógica que permite almacenar y manipular información binaria (ceros y unos). Los registros son fundamentales en la arquitectura de una computadora, ya que son responsables de guardar temporalmente datos que están siendo procesados o transferidos.

¿Cómo funcionan los Registros?

Su funcionamiento es el siguiente:

- **Almacenamiento de datos:** Cada flip-flop en un registro puede almacenar un bit de información (0 o 1). Si un registro tiene 4 flip-flops, podrá almacenar 4 bits, lo que se conoce como una "palabra" de 4 bits.
- **Operaciones básicas:** Los registros permiten realizar operaciones fundamentales sobre los datos que contienen:
 - Puesta a cero: Se puede reiniciar el contenido del registro, dejando todos los bits en 0.
 - Complemento: Cambiar todos los ceros por unos y viceversa (inversión de bits).
 - Transferencia: Mover los datos almacenados a otro registro o dispositivo.
- **Operaciones avanzadas:** Los registros pueden tener capacidades adicionales, dependiendo de su diseño:
 - Desplazamiento: Permite mover los datos hacia la derecha, izquierda, o de manera cíclica (por ejemplo, el bit más a la derecha puede pasar al extremo izquierdo).
 - Contadores: Algunos registros pueden incrementar o decrementar automáticamente los valores almacenados, actuando como contadores.
- **Lógica de control:** Para realizar estas operaciones, los registros cuentan con lógica combinacional o secuencial que interpreta señales externas (señales de control o gobernación) y decide qué operación realizar en cada momento.
- **Interacción con la memoria:** Los registros también pueden formar parte de una memoria. En este caso, se organizan en conjuntos más grandes y se añaden circuitos que permiten seleccionar qué registro específico debe leerse o escribirse, utilizando una dirección única.

¿Qué Tipos de Registros podemos encontrar?

Algunos de los registros que podemos encontrar son:

1. Registros de datos: Estos registros se utilizan para guardar números enteros o resultados de operaciones aritméticas y lógicas. En los primeros procesadores, existía un único registro de datos llamado "acumulador", donde se almacenaban temporalmente los datos para realizar cálculos. Hoy en día, los procesadores modernos tienen múltiples registros de datos que permiten realizar varias operaciones en paralelo.

Ejemplo: Un registro de datos puede almacenar temporalmente el número 5 para luego sumarlo con otro número.

2. Registros de memoria: Son registros especializados en almacenar direcciones de memoria, no datos directamente. Son fundamentales en arquitecturas como Harvard, donde las direcciones y los datos se tratan de forma separada. Estos registros permiten que el procesador sepa en qué lugar de la memoria está guardado un dato o instrucción.

Ejemplo: Si necesitas buscar un archivo en una carpeta, el registro de memoria guardaría la ubicación exacta de esa carpeta en tu computadora.

3. Registros de propósito general (GPR): Estos registros, como su nombre lo indica, se utilizan para cualquier propósito dentro del procesador. Pueden almacenar tanto datos como direcciones. Son esenciales en la arquitectura Von Neumann y son los registros más versátiles en los procesadores modernos.

Ejemplo: En un juego, un registro de propósito general podría guardar temporalmente la posición de un personaje en la pantalla.

4. Registros de coma flotante: Están diseñados específicamente para trabajar con números en formato de coma flotante (números decimales). Estos registros son esenciales en aplicaciones científicas, gráficas y cualquier tarea que requiera precisión decimal.

Ejemplo: Si estás calculando el área de un círculo (πr^2), los valores decimales como 3.14 se procesarían en registros de coma flotante.

5. Registros de constantes: Estos registros contienen valores fijos que no se pueden modificar, ya que están definidos por hardware. Se usan para tareas

específicas y suelen ser de solo lectura.

Ejemplo: Un registro de constante puede almacenar el valor de la dirección de inicio del sistema operativo, que siempre será el mismo cuando se encienda la computadora.

6. Registros de propósito específico: Son registros que almacenan información relacionada con el estado del sistema. Ejemplos de estos son:

- **Puntero de pila (stack pointer)**: Almacena la dirección de la pila de memoria que se utiliza para guardar datos temporales.
- **Registro de estado (status register)**: Indica el estado actual del procesador, como si una operación ha producido un error o si se ha generado un resultado negativo.

Ejemplo: Cuando tu computadora realiza un cálculo, el registro de estado puede señalar si el resultado es correcto o si hubo un problema, como un desbordamiento de memoria.

¿Cómo es la estructura básica de un Registro?

La estructura básica de un registro es la siguiente:



- **Celdas de almacenamiento (Flip-flops):** En el centro del gráfico se observan varias celdas identificadas como Q, Q', An, An-1, ..., A0. Cada una de estas celdas corresponde a un flip-flop, que es la unidad básica de almacenamiento de un bit (0 o 1).

Si el registro tiene n+1 flip-flops, significa que puede almacenar n+1 bits, lo que equivale a una "palabra" de n+1 bits.

- **Gobierno de entradas y salidas:** La parte superior del gráfico muestra el Gobierno de las Entradas/Salidas, que controla cómo los datos ingresan y salen del registro.

Esto incluye las líneas de entrada (para cargar datos en el registro) y las líneas de salida (para transferir datos almacenados a otro componente del sistema).

- **Gobierno de las operaciones:** En la parte inferior del gráfico está el Gobierno de las Operaciones, que controla qué tipo de operación se realiza dentro del registro. Por ejemplo:
 - Escritura: Grabar nuevos datos en el registro.
 - Lectura: Extraer datos almacenados.
 - Operaciones especiales: Desplazamientos (hacia la derecha o izquierda) o complementos de bits.
- **Lógica combinacional y secuencial asociada:** Los bloques de entrada/salida y operación están conectados a una lógica combinacional y secuencial, que coordina todas las acciones en función de señales externas (como pulsos de reloj o comandos del procesador). Esta lógica determina qué flip-flop debe activarse y qué datos deben procesarse.
- **Flujo de control:** Las flechas en el gráfico representan el flujo de datos (entradas y salidas) y las señales de control que indican al registro qué hacer en cada momento. Por ejemplo:
 - Flechas hacia arriba: Datos que entran o salen del registro.
 - Flechas hacia los flip-flops: Señales que activan operaciones específicas como "guardar" o "leer".

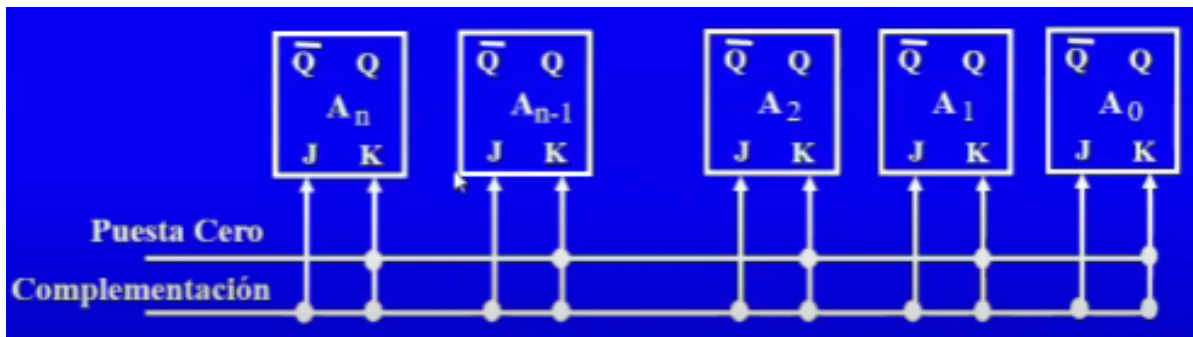
¿Qué operaciones se nos pueden presentar?

Las operaciones que pueden presentarse son:

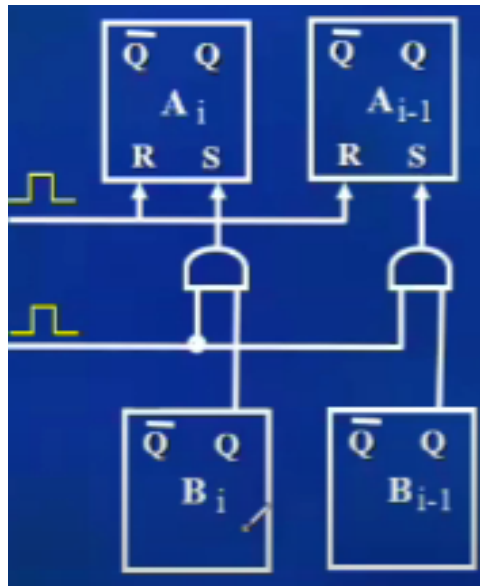
- Puesta a Cero: La operación de puesta a cero implica que todos los flip-flops del registro (cada $A_n, A_{n-1}, \dots, A_0$) establecen sus salidas (Q) en 0. Esto borra cualquier información almacenada previamente en el registro, dejándolo en un estado neutral.

Funcionamiento:

- Se activa una línea de control llamada "Puesta a Cero" (indicada en el gráfico).
- Esta señal viaja a todos los flip-flops simultáneamente.
- Al recibir esta señal, los flip-flops ejecutan un "reset", lo que garantiza que las salidas Q de todos los flip-flops sean 0, independientemente de cuál sea su estado anterior.
- Complementación: La operación de complementación cambia el estado actual de los flip-flops invirtiendo sus salidas:
 - Si un flip-flop tiene $Q=0$, después de la complementación, Q será 1.
 - Si un flip-flop tiene $Q=1$, después de la complementación, Q será 0.



- Transferencia con Puesta a Cero Previa: Esta operación asegura que antes de realizar la transferencia de datos entre registros, el registro de destino sea inicializado completamente en 0. Esto elimina cualquier contenido previo en el registro de destino para evitar interferencias.



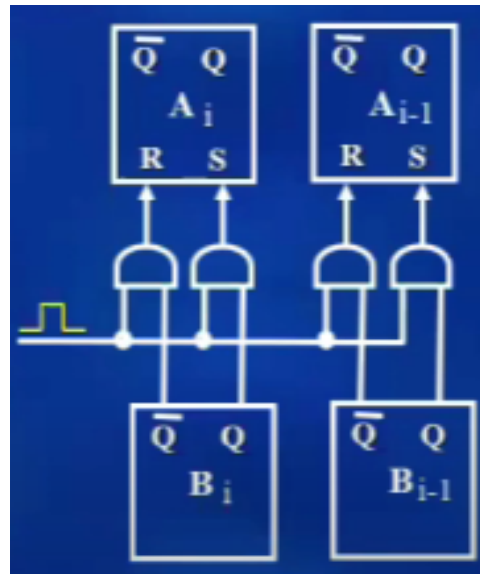
Funcionamiento:

- Se activa la señal de Puesta a Cero, que afecta al registro de destino.
- Los flip-flops del registro de destino fijan sus salidas (Q) en 0, dejando el registro preparado para recibir los datos del registro de origen.
- Una vez que el registro de destino está limpio, los datos almacenados en el registro de origen son transferidos bit a bit hacia el registro de destino a través de las líneas de conexión.

Ejemplo práctico:

Cuando un procesador necesita copiar el contenido de un registro que contiene un número binario al acumulador (registro especial que se usa para cálculos), primero limpia el acumulador y luego realiza la transferencia.

- Transferencia con Entrada Forzada (Variante 1, directa): En esta operación, el contenido del registro de origen se transfiere directamente al registro de destino sin realizar modificaciones. La entrada forzada asegura que cada flip-flop en el registro de destino adopte exactamente el valor del flip-flop correspondiente en el registro de origen.



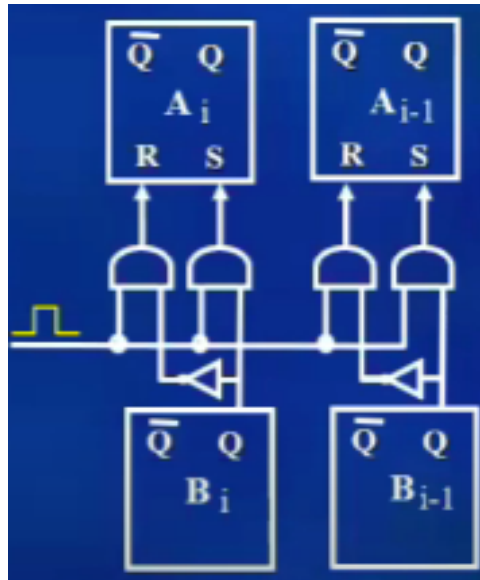
Funcionamiento:

- La señal de control activa las compuertas lógicas que conectan los flip-flops del registro de origen con los flip-flops del registro de destino.
- Cada bit del registro de origen (Q) se copia directamente al flip-flop correspondiente en el registro de destino.
- Esto se realiza en un solo ciclo de reloj si el sistema es síncrono.

Ejemplo práctico:

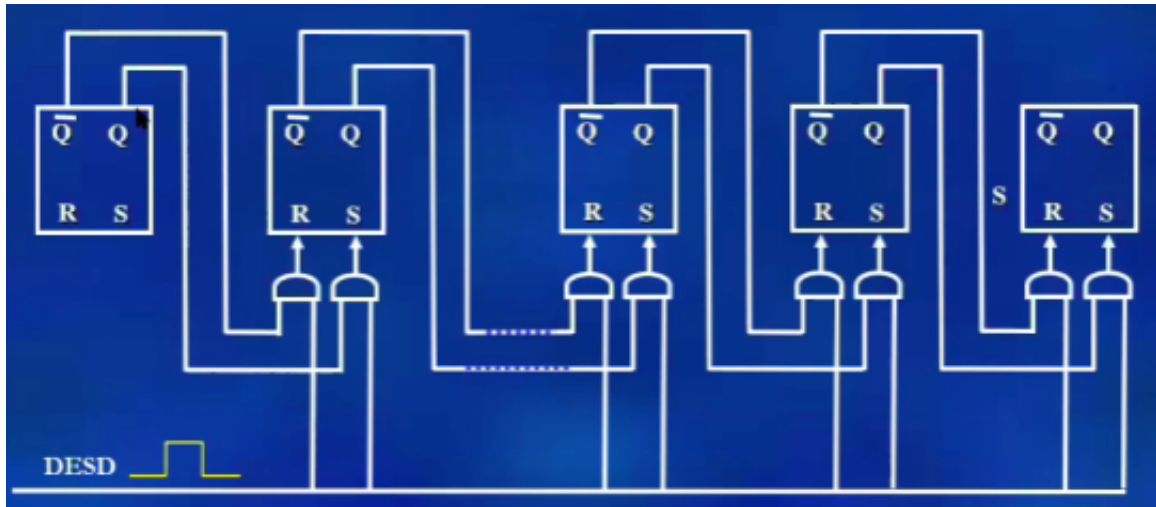
Cuando se realiza una operación de suma en la ALU, y el resultado debe ser transferido del acumulador a otro registro para ser almacenado o procesado posteriormente.

- Transferencia con Entrada Forzada (Variante 2, con inversión): En esta variante, la transferencia de datos del registro de origen al registro de destino se realiza directamente desde las salidas Q del origen, pero pasando a través de compuertas NOT. Esto significa que no se modifica la lógica del proceso en sí, sino que el diseño del circuito cambia para trabajar exclusivamente con Q en lugar de $\overline{Q}$.



Funcionamiento:

- En el circuito anterior, la transferencia se realiza directamente desde $\bar{Q}$ (la salida complementaria de cada flip-flop en el registro de origen).
- En esta variante, los datos se toman de Q , es decir, la salida normal de cada flip-flop.
- Para imitar el comportamiento de trabajar con $\bar{Q}$, cada señal de salida de Q del origen pasa a través de una compuerta NOT antes de ingresar al registro de destino.
- La compuerta NOT invierte el dato, de forma que la entrada al destino es idéntica a lo que habría sido usando directamente $\bar{Q}$.
- Desplazamiento: La operación de desplazamiento en registros es una técnica fundamental que permite mover la información almacenada en un registro hacia la derecha, izquierda o incluso en ambos sentidos, dependiendo de las señales de control y la lógica combinacional asociada.

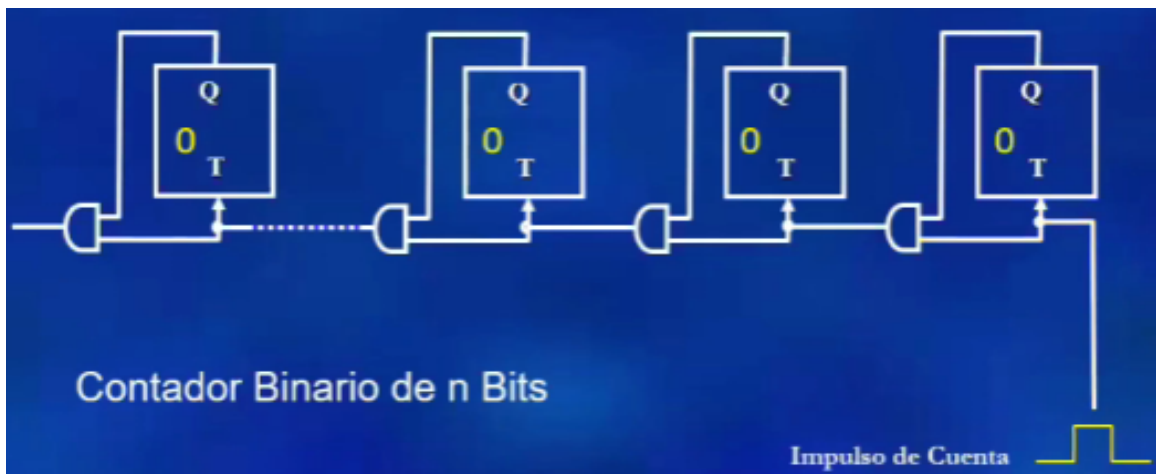


Funcionamiento:

El desplazamiento en registros mueve los bits almacenados hacia una dirección específica (derecha o izquierda). En cada movimiento, los bits cambian de posición: el más a la derecha o izquierda sale del registro, y el extremo opuesto se rellena con un valor predefinido, generalmente 0. Este proceso es controlado por una señal y sincronizado con un reloj, asegurando que los datos se muevan de forma ordenada y predecible.

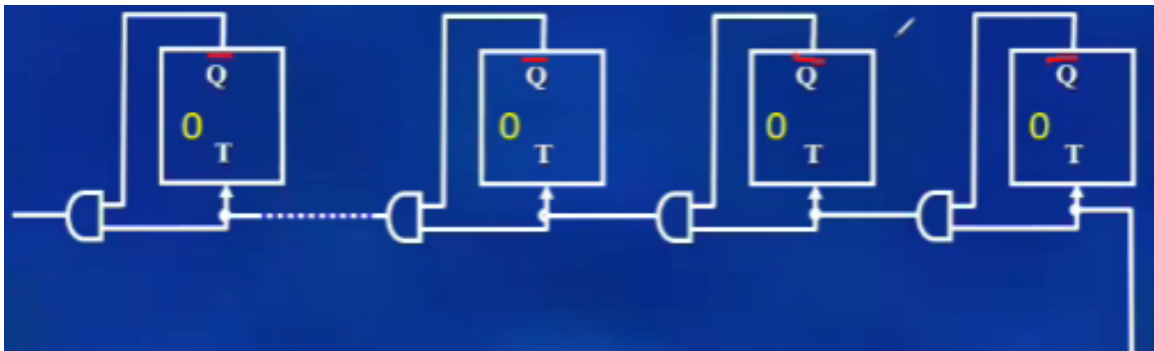
Ejemplo practico:

- Contador: Un registro contador es un circuito compuesto por flip-flops que permite incrementar (o en algunos casos decrementar) su contenido binario en una unidad por cada señal de entrada llamada **impulso de cuenta**. Es como un "reloj" que avanza de un número binario al siguiente con cada "tic".



Funcionamiento:

- Estado inicial: Todos los flip-flops están en 0, es decir, el valor del contador es 0000.
 - Primer pulso del reloj (impulso de cuenta): El primer flip-flop (Q0) cambia su estado de 0 a 1. Ahora el contador es 0001.
 - Segundo pulso: El flip-flop Q0 vuelve a cambiar, de 1 a 0. Este cambio genera un pulso para el segundo flip-flop (Q1), que ahora cambia de 0 a 1. El contador es 0010.
 - Tercer pulso: Q0 cambia de nuevo a 1, pero Q1 no cambia. El contador es 0011.
 - Cuarto pulso: Q0 vuelve a 0, y este cambio activa Q1, que pasa de 1 a 0, lo que a su vez genera un pulso para el siguiente flip-flop (Q2). Ahora el contador es 0100.
 - Y así sucesivamente: Los flip-flops trabajan en cascada. Cada vez que un flip-flop vuelve a 0, genera un pulso que activa al siguiente.
- Descontador: Un descontador es muy similar a un contador, pero con una diferencia clave, utiliza las salidas negadas ($\overline{Q}$) de los flip-flops, en lugar de las salidas normales (Q), para controlar los flip-flops siguientes.



Funcionamiento:

- Estado inicial: Supongamos un descontador de 4 bits que empieza en el valor máximo 1111 (15 en decimal).

Todos los flip-flops están en 1 inicialmente:

$Q_3 = 1, Q_2 = 1, Q_1 = 1, Q_0 = 1.$

- Primer pulso del reloj: El primer flip-flop (Q_0) recibe el pulso directamente y cambia su estado de 1 a 0.

Su salida negada ($\overline{Q_0}$) pasa de 0 a 1, pero como este cambio no afecta a los demás flip-flops, el resto permanece igual.

Resultado: 1110 (14 en decimal).

- Segundo pulso del reloj: El flip-flop Q_0 vuelve a cambiar de 0 a 1.

Este cambio en Q_0 genera un pulso en $\overline{Q_0}$, que activa al siguiente flip-flop (Q_1).

Q_1 cambia de 1 a 0, mientras que Q_2 y Q_3 permanecen igual.

Resultado: 1101 (13 en decimal).

- Tercer pulso del reloj: Q_0 cambia nuevamente de 1 a 0.

Como $\overline{Q_0}$ pasa de 0 a 1, no activa a Q_1 porque su estado ya no permite un cambio.

Resultado: 1100 (12 en decimal).

- Cuarto pulso del reloj: Q_0 cambia de 0 a 1, generando otro pulso en $\overline{Q_0}$.

Este pulso activa nuevamente a Q_1 , que cambia de 0 a 1. como $\overline{Q_1}$ pasa de 0 a 1, ahora activa al siguiente flip-flop (Q_2).

Q_2 cambia de 1 a 0, mientras que Q_3 permanece igual.

Resultado: 1011 (11 en decimal).

- Propagación sucesiva: Este proceso se repite de manera continua, Q_0 cambia con cada pulso del reloj.

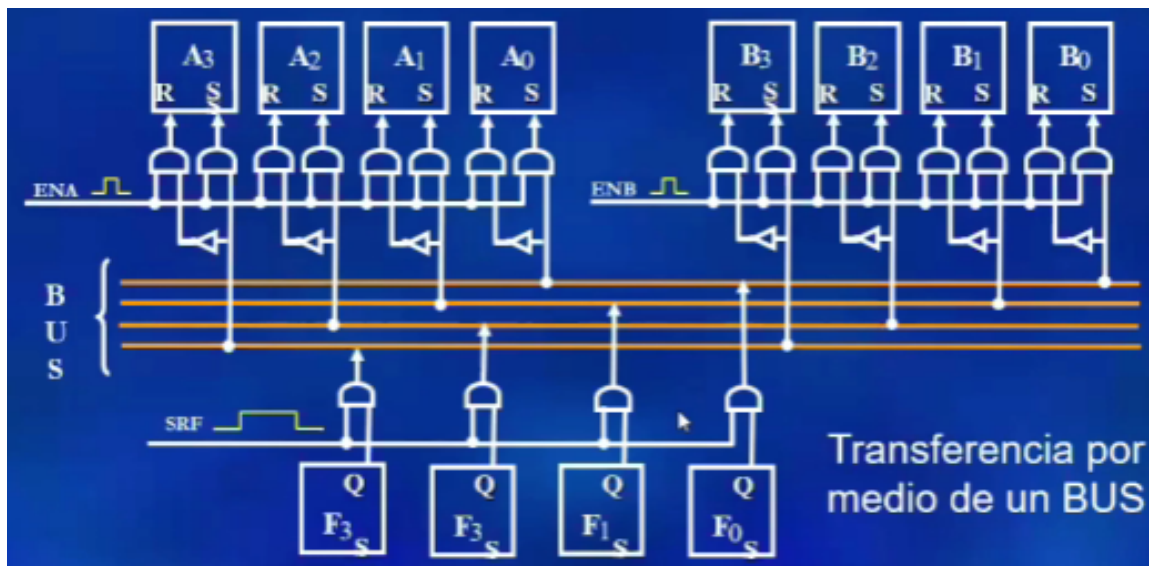
Los siguientes flip-flops (Q_1, Q_2, Q_3 , etc.) cambian su estado solo cuando las salidas negadas ($\overline{Q_i}$) de los flip-flops anteriores lo permiten.

Esto asegura que el valor binario decrezca de forma precisa y sincronizada con cada pulso del reloj.

Buses

¿Qué son los Buses?

Los buses en informática son el sistema encargado de transportar información entre distintos componentes de un computador, como procesadores, memorias, dispositivos de entrada/salida, entre otros. El término proviene del inglés y significa "transporte", lo cual refleja su función principal de interconectar y transferir datos binarios entre los registros de diferentes órganos o módulos del sistema.



¿Cómo es la Estructura de un Bus?

Un bus está compuesto por un conjunto de líneas lógicas, que son conductores eléctricos organizados en pistas metálicas (en la placa madre, por ejemplo). Cada línea puede transportar un bit (0 o 1) a la vez. Estas líneas están dispuestas para manejar tres tipos principales de datos:

- **Datos binarios:** Información que debe ser transferida entre componentes.
- **Direcciones:** Indicaciones sobre dónde debe ser enviada o desde dónde debe ser tomada la información.
- **Control:** Señales que coordinan el flujo de datos y garantizan que los componentes involucrados en la transferencia operen correctamente

¿A que hace referencia el tiempo de llenado de un Bus?

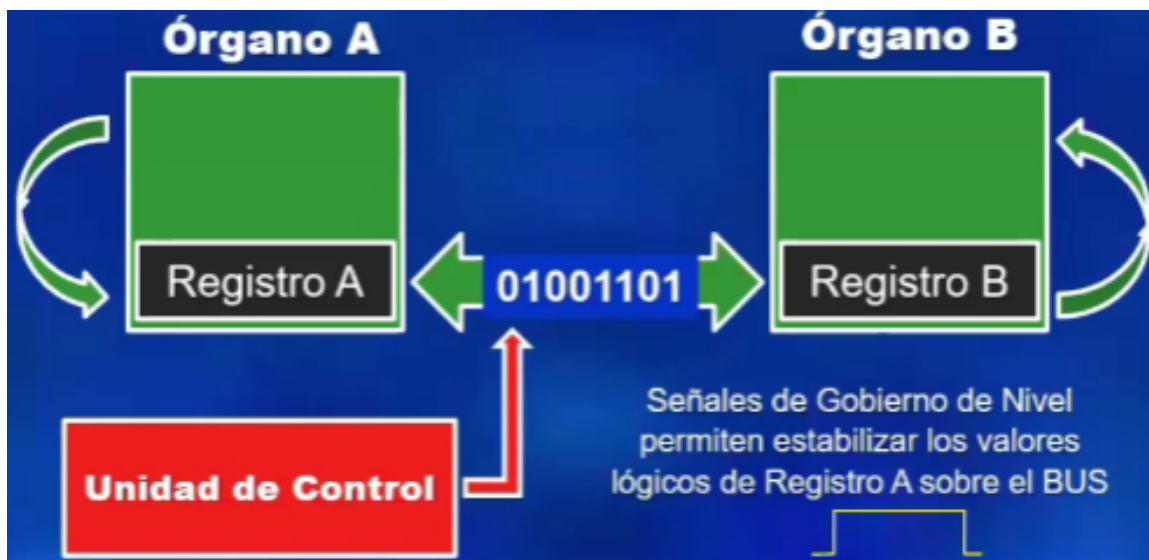
El tiempo de llenado del bus es el intervalo de tiempo necesario para cargar los datos en el bus desde un componente emisor (como un registro o una memoria), antes de que estos puedan ser transmitidos hacia un componente receptor (como

otro registro, la memoria o un periférico). Este tiempo incluye la activación de las señales de control y la propagación de los datos a través de las líneas del bus.

¿Cómo se relacionan los Buses con la Unidad de Control?

La unidad de control es quien decide cuándo y cómo los componentes del sistema (como memoria, procesador, o dispositivos de entrada/salida) acceden a los buses.

Lo hace mediante señales de gobierno de nivel, las cuales son las encargadas de habilitar, coordinar y regular el acceso a los buses por parte de los diferentes componentes del sistema (como memoria, procesador, y periféricos). Estas señales aseguran que solo el componente autorizado en ese momento pueda usar el bus, evitando conflictos y garantizando una transferencia de datos ordenada.

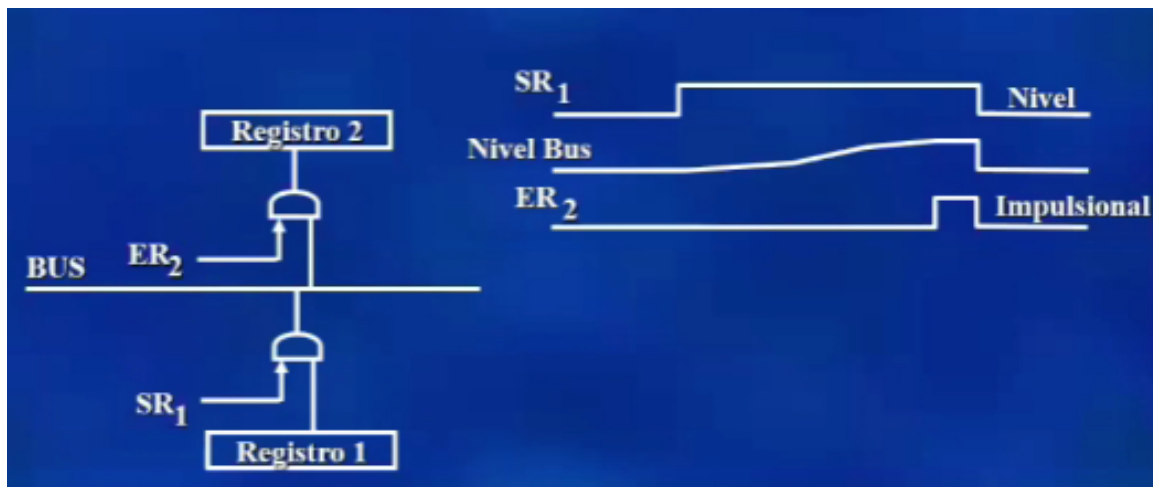


¿Cómo se relacionan las señales de Nivel e Impulsionales con los órganos Emisores y Receptores?

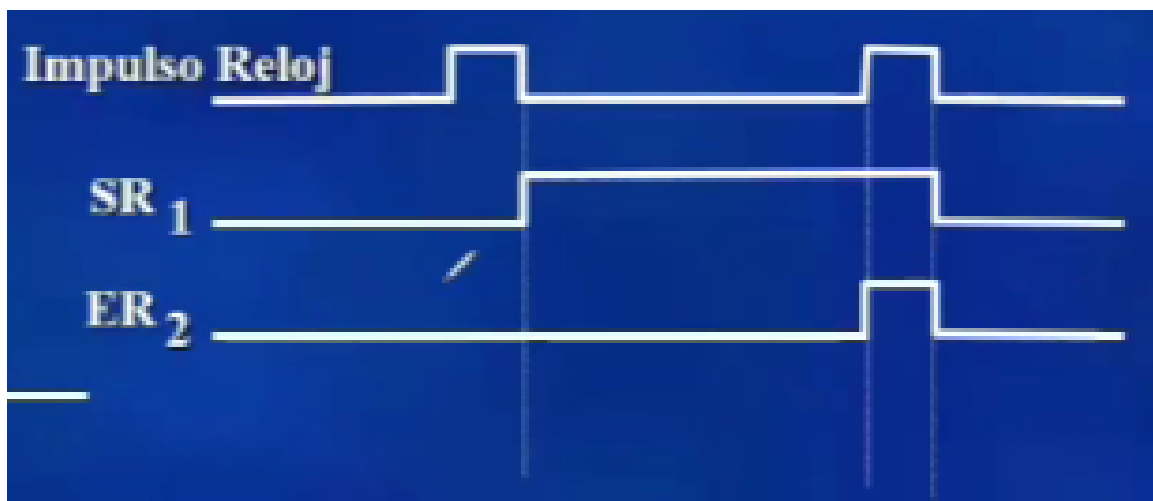
La SRF (Señal de nivel) gobierna el bus y garantiza la estabilidad de los datos durante toda la transferencia. Actúa sobre el emisor y el bus, asegurando que las líneas de comunicación estén disponibles y activas mientras se realiza la operación de envío de datos. Su carácter constante permite mantener la validez de los datos hasta que sean capturados por el receptor.

Y la ENA (Señal impulsional) activa al receptor en un momento puntual, permitiendo que capture los datos del bus de manera precisa y eficiente. ENA no

necesita mantenerse activa durante toda la transferencia, ya que su función es habilitar temporalmente al receptor en el instante adecuado.



Se debe tener en cuenta que en los sistemas digitales sincrónicos, las señales impulsionales no deben ocurrir entre impulsos de reloj porque podrían desestabilizar el sistema, generar inconsistencias o pérdida de datos, y romper la sincronización establecida por el reloj, es por ello que cualquier señal impulsional debe alinearse estrictamente con los impulsos del reloj.



ABACUS

¿Qué es ABACUS?

La Abacus está diseñada como una máquina síncrona, lo que significa que todas sus operaciones están controladas y sincronizadas por un reloj central. Este reloj regula el flujo de información entre los diferentes componentes del sistema, permitiendo que cada operación ocurra en momentos específicos y de manera ordenada.

El propósito principal de la Abacus es realizar cálculos en base al valor que se encuentra almacenado en el registro Acumulador de la Unidad Aritmético Lógica (UAL). Este registro actúa como un espacio temporal para los datos, ya que es donde se almacenan los resultados intermedios de las operaciones.

¿Con que trabaja esta arquitectura?

La arquitectura utiliza un registro de instrucción que contiene dos partes principales:

- **Código de operación:** Define la operación que debe ejecutarse (por ejemplo, sumar, restar, cargar un valor, etc.).
- **Dirección del operando:** Especifica dónde se encuentra el operando necesario para realizar la operación, ya sea en memoria o en algún registro específico.

Esto significa que cada instrucción tiene un formato que no solo describe lo que debe hacerse, sino también los datos con los que debe trabajarse.

¿Qué rol cumple el reloj en esta arquitectura?

El reloj es un componente fundamental en la Abacus. Produce un batido regular (Θ) que sincroniza todos los componentes. Cada "batido" del reloj representa un ciclo básico de memoria. En esta arquitectura, el ciclo de memoria se define como 2Θ , es decir, dos pulsos consecutivos del reloj.

El uso de un ciclo base de 2Θ tiene implicaciones en cómo se temporizan las operaciones en el sistema, permitiendo que cada componente complete su tarea antes de que comience la siguiente etapa.

¿Cómo se organizan los buses en ABACUS?

La Abacus cuenta con dos buses principales que transportan información entre sus componentes:

- Bus S (Bus de Direcciones): Este bus transporta direcciones dentro del sistema y está compuesto por m bits, suficientes para direccionar todos los

registros de la memoria. Sus principales funciones son:

1. **Ubicación de Instrucciones:** Permite al Registro P (Contador de Programa) transferir la dirección de la próxima instrucción a la Memoria, para que esta sea leída y enviada al Registro I.
2. **Búsqueda de Operandos:** Durante la ejecución de una instrucción, el campo D (Dirección del Operando) del Registro de Instrucción (I) transfiere su valor a través del Bus S hacia la Memoria, para buscar el operando necesario.
3. **Selección de Memoria:** Trabaja junto con el Registro S (Registro de Selección de Memoria) para identificar la posición exacta en la memoria que se debe leer o escribir.

En resumen, el Bus S sirve para transferir direcciones, asegurando que las instrucciones y los datos correctos sean accedidos en el momento oportuno.

- **Bus M (Bus de Datos e Instrucciones):** Este bus transporta datos y palabras de instrucción entre las diferentes unidades del sistema. Está compuesto por n bits, equivalentes al tamaño de las palabras manejadas por la memoria y los registros. Sus principales funciones son:

1. **Transferencia de Instrucciones:** Una vez que el Bus S localiza la dirección de una instrucción en la Memoria, el Bus M transporta esa instrucción al Registro I (Registro de Instrucción) para que sea decodificada.
2. **Intercambio de Datos:** Durante la ejecución de una instrucción, el Bus M transfiere los datos desde:
 - La Memoria al Registro M (Registro de Memoria): Para proporcionar operandos que serán utilizados en cálculos.
 - La Unidad Aritmético-Lógica (UAL) a la Memoria: Para guardar los resultados de las operaciones realizadas.
3. **Comunicación entre Componentes:** Vincula la Memoria, la UAL, y otros elementos del sistema, facilitando el flujo de datos durante la ejecución de operaciones.

- Fase de Decodificación de Instrucción

1. **Separación de Componentes de la Instrucción:**

- El Registro I divide la instrucción en dos partes:
 - Código de Operación (CO): Especifica la operación a realizar (suma, resta, carga, etc.).
 - Dirección del Operando (D): Indica la ubicación en memoria donde se encuentra el operando necesario.

2. **Acción de la Unidad de Control:**

- La Unidad de Control interpreta el CO y genera las señales de gobierno necesarias para que todos los componentes colaboren en la ejecución de la instrucción.

- Fase de Búsqueda del Operando

1. **Transferencia de Dirección del Operando:**

- El campo D del Registro I transfiere la dirección del operando al Registro S a través del Bus S.

2. **Acceso a la Memoria:**

- El Registro S selecciona la posición del operando en la memoria.
- La memoria encuentra el operando en la ubicación especificada.

3. **Transporte del Operando:**

- El operando encontrado en la memoria es enviado al Registro M (Registro de Memoria) a través del Bus M.

- Fase de Ejecución de la Instrucción

1. **Operación en la UAL:**

- La UAL (Unidad Aritmético-Lógica) toma dos valores:
 - El contenido del Registro AC (Acumulador).
 - El operando almacenado en el Registro M.
- Según el CO, realiza la operación solicitada (suma, resta, etc.).

2. **Almacenamiento del Resultado:**

- El resultado de la operación se escribe de nuevo en el Registro AC, reemplazando el contenido anterior.
- Si es necesario, el resultado también puede ser enviado a la memoria a través del Registro M.
- Incremento del Contador de Programa

1. **Preparación para la Siguiente Instrucción:**

- Una vez completada la instrucción actual, el Registro P se incrementa automáticamente en uno.
- Este incremento asegura que la próxima instrucción se buscará en la posición siguiente de la memoria.

Tamaño de Palabra y Cantidad de Palabras

¿A que nos referimos "Tamaño de palabra" y "Cantidad de Palabras"?

Nos referimos a lo siguiente:

- Tamaño de Palabra: El "tamaño de palabra" se refiere a la cantidad de bits que se manejan de manera unitaria en las operaciones de la arquitectura. Es decir, es la longitud en bits de la unidad de datos más básica que la computadora puede procesar en un solo ciclo de operación.

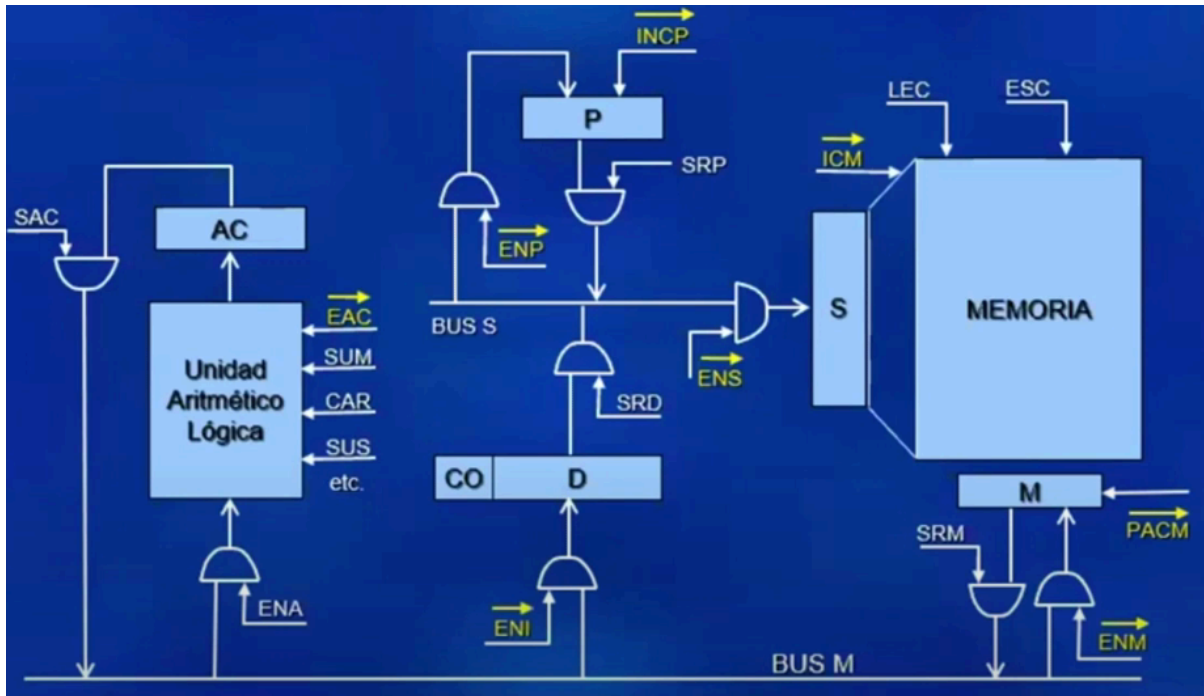
Por ejemplo, si el tamaño de palabra es de 8 bits (1 byte), entonces cada instrucción, cada operando, y cada operación aritmética o lógica de la Unidad Aritmético-Lógica (UAL) se realizará sobre bloques de 8 bits.

- El tamaño de palabra afecta directamente a:
 - **La UAL y el Registro AC (Acumulador):** Estos componentes deben ser capaces de procesar y almacenar palabras del tamaño específico. Si el tamaño de palabra es de 8 bits, tanto el acumulador como la UAL operarán sobre datos de 8 bits.
 - **El Registro M y la Memoria:** La memoria se organiza en unidades de palabras, y cada palabra tiene un tamaño específico. El Registro M, que intercambia datos con la memoria, debe ser capaz de recibir y almacenar una palabra completa de la memoria.

- **El Bus M (Bus de Datos e Instrucciones):** El ancho del Bus M debe coincidir con el tamaño de palabra, ya que transporta las palabras de datos e instrucciones entre los componentes.
- Cantidad de Palabras en Memoria: La "cantidad de palabras en memoria" se refiere a cuántas unidades de palabras de un tamaño específico pueden almacenarse en la memoria principal. Esto define la capacidad de la memoria.
Si se tiene una memoria de 2^m palabras, significa que la memoria puede contener 2^m bloques de datos, cada uno del tamaño de una palabra.
 - La cantidad de palabras en memoria afecta a:
 - **El Registro S (Registro de Selección) y el Registro P (Contador de Programa):** El tamaño de estos registros está directamente relacionado con la cantidad de palabras que la memoria puede manejar. Si la memoria puede almacenar 2^m palabras m bits, entonces los registros de selección y el contador de programa deben tener al menos para poder direccionar cualquier palabra en la memoria.
 - **El Bus S (Bus de Direcciones):** El ancho de este bus debe ser suficiente para transportar las direcciones de las palabras en memoria. Si la memoria tiene 2^m palabras, el Bus S debe tener una capacidad de m bits para transferir esas direcciones.

Señales de Gobierno de ABACUS

¿Qué señales de Gobierno existen?



Tenemos las señales:

- De Nivel:

- **SAC:** Permite que el contenido del Acumulador se establezca sobre el Bus M, con el objetivo de mover el resultado del Acumulador y grabarlo en la Memoria.
- **ENA:** Permite estabilizar el contenido del Bus M sobre la ALU.
- **SUM, CAR, SUS:** Son las operaciones que se realizan en la ALU, y son de nivel porque, por lo general, no les alcanza un impulso para propagar el calculo completo dentro de la ALU.
- **SRD:** Permite mover el contenido de la Dirección del Operando (D) sobre el Bus S, estabilizándolo para poder buscar el operando.

También permite mover el contenido de D, cuando D es una instrucción de salto, lo mueve al Bus S y a través de ENP puede variar la dirección de la próxima instrucción.

- **SRP:** Permite que el contenido del contador de programa se establezca sobre el Bus S.
- **LEC:** Permite leer un dato en la memoria.

- **ESC:** Permite escribir un dato en la memoria.
Las señales de lectura (LEC) y escritura (ESC) son de nivel porque necesitan mantenerse activas durante varios ciclos de reloj para garantizar que la operación de acceso a la memoria se complete correctamente, permitiendo que los buses internos estabilicen los datos y direcciones requeridas. Sin estas señales de nivel, el acceso a la memoria podría ser inconsistente o erróneo.
- **SRM:** Lo que hace es, a través de una lectura, la palabra leída la mueve al Registro Palabra Memoria y la estabiliza sobre el Bus M. Cabe aclarar que este registro puede contener un operando o una instrucción.
- Impulsionales:
 - **EAC:** Permite mover el resultado de la ALU al Acumulador.
 - **ENP:** Permite copiar el valor del Bus S y llevarlo al Contador de Programa para poder realizar una instrucción específica, salto condicional o incondicional.
 - **INCP:** Permite que el contenido del Contador de Programa se incremente en uno.
 - **ENI:** Permite copiar el contenido del Bus M directamente al Registro de Instrucción para que la Unidad de Control tome el gobierno la decodificación de la instrucción y actúe en consecuencia.
 - **ENS:** Carga el contenido del Bus S al Registro de Selección de Memoria.
 - **ICM:** Es una señal de aviso, activa el decodificador de direcciones, y marca el inicio del ciclo de memoria.
 - **PACM:** Aparece por una cuestión tecnológica, si el registro M requiere un acerado rápido es necesario mandar esta señal para poner en 0 dicho registro.
 - **ENM:** Copia el contenido del Bus M en el Registro Palabra Memoria para guardar seguramente un resultado en la memoria. Dicho resultado viene del Acumulador.